

MSc INDIVIDUAL PROJECT

MSc HUMAN AND BIOLOGICAL ROBOTICS

IMPERIAL COLLEGE LONDON

DEPARTMENT OF BIOENGINEERING

---

**Multimodal Teleoperation of a Wearable Dual-Arm  
Supernumerary Robotic System: Master Interface  
Design, Safety Architecture, and a Workspace  
Characterisation**

---

*Author:*  
Gaus Sayyad  
CID: 06053822

*Supervisor:*  
Dr. Dandan Zhang

*Co-Supervisor:*  
Dr. Etienne Burdet

August 8, 2026

CONFIRM: title with Dr. Zhang. Option 2 of three was accepted pending supervisor confirmation; the alternatives and their trade-offs are in `thesis_report/DIVERGENCE.md` Part D.

CONFIRM: word limit and submission format with Dr. Zhang / Prof. Burdet. The 12,000–15,000 word target used here is not authoritative.

FIGURE: Imperial College logo: run `epstopdf title/logo.eps` once ghostscript is available, then restore the `includegraphics` in `title/title.tex`.

## Abstract

A supernumerary robotic limb is mounted on a person, so its base moves, is soft, and cannot be commanded to hold still. This thesis reports a teleoperation system for two seven-degree-of-freedom arms carried on a worn backpack frame and commanded from an instrumented mannequin master, built on two premises: that commanding fourteen joints through a body-worn master is a usability problem rather than a plumbing one, and that a master routing fourteen analog channels through rotating joints will lose channels, so losing them must be survivable.

The command mapping takes each component of position from whichever sensor observes it most directly, with elevation from the gravity vector, azimuth from a single shoulder joint, and radius from a forward-kinematic magnitude insensitive to roll errors. Channel health is classified per channel over distinct sensor updates, validation is scoped to the channels the active mode consumes, and the cost of each reduction is measured on 20 430 recorded frames. With no well-conditioned channels the system reports position unavailable rather than extrapolating one.

Measurements on the physical master found six of fourteen channels usable and reversed two long-standing verdicts, both of which had come from computing statistics across transmitted rows rather than distinct updates. Vertical and fore-and-aft command track correctly on both arms; lateral does not, and a rotation-invariant test shows the measured directions are not an orthogonal triad, so no axis map can correct them. Fourteen of fourteen injected faults were handled, including two whose diagnosis required correcting the observable a safety mechanism was keyed on.

The workspace characterisation returned a negative result with a quantified mechanical remedy: under the as-built mount the two arms share no reachable cell of a sixty-three-cell frontal grid, with a 0.25 m dead band between the reachable sets, which blocks inter-arm handover for geometric reasons no control or planning change can address. A buildable mount modification raising the shared count to twenty-two is quantified and deliberately not applied.

No result was validated on a physical arm. The blockers to a two-person study are enumerated, and the first is two soldered joints.

## Acknowledgements

WRITE: acknowledgements.

# Contents

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                       | <b>8</b>  |
| 1.1      | Two design premises                                                       | 8         |
| 1.1.1    | Commanding fourteen joints is a usability problem, not a plumbing problem | 8         |
| 1.1.2    | A body-worn master will lose channels, so losing them must be survivable  | 9         |
| 1.2      | Objectives                                                                | 9         |
| 1.3      | Challenges                                                                | 9         |
| 1.4      | Contributions                                                             | 10        |
| 1.5      | Thesis structure                                                          | 11        |
| <b>2</b> | <b>Background</b>                                                         | <b>12</b> |
| 2.1      | Supernumerary robotic limbs                                               | 12        |
| 2.2      | The operator and the wearer as different people                           | 13        |
| 2.3      | Master interfaces for dissimilar kinematics                               | 13        |
| 2.4      | Shared autonomy                                                           | 14        |
| 2.5      | Measuring embodiment, workload and trust                                  | 14        |
| 2.6      | The gap                                                                   | 15        |
| <b>3</b> | <b>The master arm</b>                                                     | <b>16</b> |
| 3.1      | Mechanical chain                                                          | 16        |
| 3.2      | Electronics                                                               | 16        |
| 3.2.1    | The A4 and A5 collision                                                   | 19        |
| 3.3      | The inertial measurement driver                                           | 19        |
| 3.3.1    | Scale factors                                                             | 20        |
| 3.4      | Signal conditioning                                                       | 20        |
| 3.4.1    | Potentiometers                                                            | 20        |
| 3.4.2    | A start-up transient in the filter initialisation                         | 21        |
| 3.4.3    | A candidate explanation for the observed update rate                      | 21        |
| 3.4.4    | Force-sensitive resistors                                                 | 22        |
| 3.4.5    | Buttons                                                                   | 22        |
| 3.5      | The serial frame                                                          | 22        |

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| 3.6      | Summary                                                                | 23        |
| <b>4</b> | <b>How the control approach reached its present form</b>               | <b>24</b> |
| 4.1      | Phase 1: joint-space control in simulation                             | 24        |
| 4.1.1    | Why joint-space control was abandoned                                  | 25        |
| 4.2      | Phase 2: the move to ROS 2                                             | 25        |
| 4.2.1    | What was kept from simulation                                          | 26        |
| 4.3      | Phase 3: from Cartesian control to a reduced-degree-of-freedom command | 26        |
| 4.3.1    | Alternatives considered at this branch                                 | 26        |
| 4.4      | Summary of the sequence                                                | 27        |
| <b>5</b> | <b>System as built</b>                                                 | <b>28</b> |
| 5.1      | Architecture                                                           | 28        |
| 5.2      | From master pose to arm command                                        | 29        |
| 5.2.1    | Spherical decomposition                                                | 29        |
| 5.2.2    | Anchoring, scaling and indexing                                        | 29        |
| 5.3      | Graceful degradation                                                   | 30        |
| 5.3.1    | Channel health                                                         | 30        |
| 5.3.2    | Mode-scoped validation                                                 | 30        |
| 5.3.3    | The degradation ladder                                                 | 31        |
| 5.3.4    | Reduced-degree-of-freedom fallback                                     | 31        |
| 5.4      | Control modes                                                          | 32        |
| 5.4.1    | Why the transport for VR is USB rather than the network                | 32        |
| 5.4.2    | Why the voice mode uses a grammar and not a language model             | 32        |
| 5.4.3    | Why not an end-to-end vision-language-action policy                    | 37        |
| 5.5      | Safety architecture                                                    | 37        |
| 5.5.1    | Collision avoidance in three stages                                    | 37        |
| 5.5.2    | Blocking must be observable                                            | 37        |
| 5.5.3    | Expiry is not permission                                               | 37        |
| 5.5.4    | Emergency stop                                                         | 38        |
| 5.6      | Verification methodology                                               | 38        |
| 5.7      | Calibration                                                            | 41        |
| <b>6</b> | <b>Results</b>                                                         | <b>44</b> |
| 6.1      | Channel health                                                         | 44        |
| 6.1.1    | Progressive degradation between two dates                              | 45        |
| 6.2      | What the degradation architecture costs                                | 45        |
| 6.2.1    | Which channels a repair should target                                  | 45        |
| 6.2.2    | What the incoherent channels inject                                    | 46        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 6.2.3    | The pipeline manufactures no motion of its own . . . . .                 | 47        |
| 6.3      | Command mapping . . . . .                                                | 47        |
| 6.3.1    | Axis correctness . . . . .                                               | 47        |
| 6.3.2    | The lateral axis remains wrong, and the axis map cannot fix it . . . . . | 47        |
| 6.3.3    | Indexing . . . . .                                                       | 48        |
| 6.4      | Safety . . . . .                                                         | 48        |
| 6.4.1    | Fault injection . . . . .                                                | 48        |
| 6.4.2    | Dead-man timing . . . . .                                                | 49        |
| 6.4.3    | An emergency stop that was four seconds late . . . . .                   | 49        |
| 6.5      | Grasping . . . . .                                                       | 49        |
| 6.5.1    | Top-down grasping has a lateral boundary . . . . .                       | 49        |
| 6.6      | The wrist cannot be aligned under teleoperation . . . . .                | 50        |
| 6.7      | Task-set verification . . . . .                                          | 50        |
| 6.8      | Performance figures . . . . .                                            | 51        |
| 6.9      | What is not measured . . . . .                                           | 51        |
| <b>7</b> | <b>Workspace characterisation and its consequences</b>                   | <b>53</b> |
| 7.1      | Continuous reachability from the home pose . . . . .                     | 53        |
| 7.2      | The two arms share no reachable workspace . . . . .                      | 53        |
| 7.2.1    | What this does and does not establish . . . . .                          | 55        |
| 7.3      | The fix is buildable . . . . .                                           | 55        |
| 7.4      | One boundary, two independent constraints . . . . .                      | 55        |
| 7.5      | Consequences for the task set . . . . .                                  | 56        |
| <b>8</b> | <b>Discussion</b>                                                        | <b>58</b> |
| 8.1      | What the degradation architecture bought, and what it did not . . . . .  | 58        |
| 8.2      | The reduced-degree-of-freedom premise, assessed . . . . .                | 58        |
| 8.3      | The workspace result and what it costs the study . . . . .               | 59        |
| 8.4      | Comparison against the planning report's targets . . . . .               | 59        |
| 8.5      | Threats to validity . . . . .                                            | 59        |
| 8.5.1    | Simulation is not the arm . . . . .                                      | 59        |
| 8.5.2    | Latency figures are lower bounds . . . . .                               | 60        |
| 8.5.3    | The instrument failure rate . . . . .                                    | 60        |
| 8.5.4    | The one-stack constraint . . . . .                                       | 60        |
| 8.6      | What stands between this system and a two-person study . . . . .         | 60        |
| <b>9</b> | <b>Conclusion</b>                                                        | <b>61</b> |
| 9.1      | Against the objectives . . . . .                                         | 61        |
| 9.2      | What was not achieved . . . . .                                          | 61        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| 9.3      | Future work, in the order it should be done . . . . . | 62        |
| 9.4      | Closing . . . . .                                     | 62        |
| <b>A</b> | <b>Master arm firmware</b>                            | <b>63</b> |
| A.1      | Pin assignment as declared . . . . .                  | 63        |
| A.2      | Signal conditioning as implemented . . . . .          | 63        |
| A.3      | Frame construction . . . . .                          | 64        |
| <b>B</b> | <b>Supporting records</b>                             | <b>66</b> |

# List of Figures

|      |                                                    |    |
|------|----------------------------------------------------|----|
| 2.1  | Bracing supernumerary limbs (reproduced)           | 12 |
| 2.2  | Policy blending arbitration (reproduced)           | 14 |
| 2.3  | The two-person configuration                       | 15 |
| 3.1  | Master kinematic chain                             | 17 |
| 3.2  | Master arm CAD                                     | 17 |
| 3.3  | Master arm as built                                | 18 |
| 3.4  | Master arm electronics                             | 18 |
| 5.1  | Package architecture                               | 28 |
| 5.2  | Degradation ladder                                 | 31 |
| 5.3  | Data path: direct teleoperation from the mannequin | 33 |
| 5.4  | Data path: direct teleoperation from VR            | 34 |
| 5.5  | Data path: shared autonomy                         | 35 |
| 5.6  | Data path: full autonomy from voice                | 36 |
| 5.7  | Experimental design                                | 39 |
| 5.8  | Session timeline                                   | 40 |
| 5.9  | Calibration procedure                              | 42 |
| 5.10 | The model at home                                  | 43 |
| 6.1  | Channel health                                     | 45 |
| 6.2  | Progressive channel degradation                    | 46 |
| 6.3  | The lateral constraint                             | 48 |
| 6.4  | Operations console                                 | 51 |
| 7.1  | Continuous reachability from home                  | 54 |
| 7.2  | The measured dead band                             | 54 |
| 7.3  | Mount parameter sweep                              | 55 |
| 7.4  | Two constraints, one boundary                      | 56 |
| 7.5  | Compliant carrier geometry                         | 57 |

# List of Tables

|     |                                                                                                                                                                             |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Pin assignment. Sixteen analog channels are required and sixteen are available once the two occupied by the default I <sup>2</sup> C bus are excluded. . . . .              | 16 |
| 3.2 | Inertial sensor scaling at power-on defaults. . . . .                                                                                                                       | 20 |
| 3.3 | Filter output after reset, simulated from Eq. (3.1) with a constant input of 2000 counts and the seed used in <code>setup()</code> . . . . .                                | 21 |
| 3.4 | Serial frame, in transmission order. Fields are comma-separated; the frame ends with a newline. Buffer is 460 bytes against a typical frame of roughly 285. . . . .         | 23 |
| 4.1 | Development sequence, with the reason each stage ended. . . . .                                                                                                             | 27 |
| 5.1 | Operating modes. Autonomy runs slower than teleoperation by design: nobody is holding a master, so the only bound on a wrong motion is how long it takes to happen. . . . . | 32 |
| 5.2 | Blocker states. Expiry means the asserting loop stopped running, so the guarded condition is unknown rather than absent. . . . .                                            | 38 |
| 6.1 | Per-channel verdicts, computed over distinct sensor updates. Baseline stored at <code>recordings/baselines/channels_2</code>                                                |    |
| 6.2 | Reach observability from the surviving channels. . . . .                                                                                                                    | 46 |
| 6.3 | Commanded end-effector motion contributed by the incoherent channels alone, with the operator's input held constant. . . . .                                                | 46 |
| 6.4 | Dead-man behaviour by failure mode. . . . .                                                                                                                                 | 49 |
| 6.5 | Five-task verification. Both arms verified at home; ten repeats per pose; short-circuit on first failure.                                                                   | 50 |
| 6.6 | Measured rates and latencies. All against mock hardware. . . . .                                                                                                            | 51 |
| 7.1 | Both-arm overlap under three test conditions . . . . .                                                                                                                      | 54 |
| B.1 | Supporting records and where they live. . . . .                                                                                                                             | 66 |

# Chapter 1

## Introduction

A supernumerary robotic limb is an extra arm worn on the body, working alongside the wearer’s own limbs rather than replacing or rehabilitating them [30, 25]. The wearer keeps full natural function, which separates these devices from prostheses. What separates them from fixed manipulators is more consequential for control: the base of the robot is a person. It accelerates, sways and steps. It is soft and irreplaceable. It cannot be told to hold still.

This thesis reports work on the MUlti-limb Virtual Environment, known within Imperial College London as the Dr. Octopus platform [9], which mounts two Kinova Gen3 seven-degree-of-freedom arms [14] on a backpack frame. The problem addressed is the command interface. Two redundant seven-degree-of-freedom arms present fourteen commandable joints, and a person has two hands and a limited attention budget. Something has to close that gap.

### 1.1 Two design premises

The work rests on two premises that were adopted early and shaped everything downstream. Both are stated here because the rest of the thesis only makes sense if they are visible from the start.

#### 1.1.1 Commanding fourteen joints is a usability problem, not a plumbing problem

The naive interface exposes every joint. It is also unusable. A body-worn master that reproduces the slave’s full kinematic chain requires the operator to coordinate seven degrees of freedom per arm through finger motion, with no force feedback and no view of the joint the arm is actually in. Earlier work on miniature physical masters makes the case for morphological similarity between master and slave [7], and that case is accepted here. What does not follow is that every degree of freedom of the slave must be commanded independently by the operator.

The architecture adopted instead splits the command by what each source measures well. Position comes from three or four well-conditioned potentiometer channels resolved into a spherical form about the master’s shoulder. Orientation comes from the inertial measurement unit at the wrist, which observes gravity directly and therefore needs no integration and no calibration against an absolute reference. What is left over, principally the wrist orientation required to grasp a particular object, is supplied by autonomy or is not commanded at all.

This is a reduced-degree-of-freedom design by intent. It is the reason the system uses a spherical decomposition rather than forward kinematics from seven joint angles, and the reason its default orientation mode pins the wrist rather than tracking it. Both choices cost capability, and both are quantified in [Chapter 6](#).

### 1.1.2 A body-worn master will lose channels, so losing them must be survivable

The master carries fourteen analog channels routed through seven rotating joints per arm. Connectors fatigue. Wires flex through joints that turn. Over the life of any such device some fraction of its channels will stop reporting usable values, and the fraction will grow. A design whose behaviour is undefined when one channel fails is not a design that has been finished.

The system therefore treats channel loss as an operating condition rather than a fault. Each channel carries a health verdict computed from its own recent history. Validation is scoped to the channels the active position mode consumes, so a dead channel that the mode does not read cannot reject a frame. Individual channels can be disabled at run time, and the resulting loss of capability is published rather than inferred. When enough channels are lost that position can no longer be estimated, the system reports position unavailable instead of extrapolating one.

That last behaviour is the design's centre of gravity. On a manipulator bolted beside a person's head, a fabricated position is worse than no position, and the difference between them has to be visible from outside the process.

## 1.2 Objectives

The planning report framed this project around a haptic master and slave pair with physical springs at the master and virtual impedance at the slave. The delivered system diverged from that plan, for reasons set out in [Chapter 8](#) and recorded in full in the accompanying divergence analysis.

**CONFIRM:** whether the divergence analysis is submitted as an appendix or remains a supporting file.

The objectives below are the revised set, aligned to what was built and measured.

1. To design and build an instrumented mannequin master reproducing the Gen3 morphology at reduced scale, and to characterise its sensing against recorded data rather than against specification.
2. To derive a mapping from master pose to slave end-effector command that degrades predictably as sensing channels are lost, and to quantify what each loss costs.
3. To implement a safety architecture appropriate to a manipulator mounted beside a person, and to validate it by injecting faults rather than by inspecting code.
4. To characterise the reachable workspace of the mounted configuration and determine which manipulation tasks it can support.
5. To establish a verification methodology producing traceable evidence for every claim, and to apply it across the task set.
6. To identify, by measurement, what must be resolved before hardware validation and a user study become possible.

## 1.3 Challenges

Four difficulties consumed most of the effort.

*The master and the slave are not the same shape.* The master reproduces the Gen3's joint sequence but not its proportions. Its links sum to 0.272 m against the arm's 0.902 m reach, a ratio of roughly three to one, so joint-to-joint correspondence is unavailable. Working in task space instead raises a second problem, which is that the two chains reach different sets of poses. The resolution was to map displacement of the master's tip onto displacement of the slave's end effector about an anchor that can be re-latched, described in [Chapter 5](#).

*Two coordinate conventions meet at the driver.* The manufacturer's controller reports joint positions in degrees over  $[0, 360)$ ; the middleware uses radians over  $(-\pi, \pi]$ . The failure this invites is not a small error. An angle of  $350^\circ$  read as  $+350$  rather than  $-10$  commands very nearly a full revolution of a joint standing next to a person. The conversion was isolated in one module and tested at the seams.

*Calibration references must survive a power cycle without producing a step.* The potentiometers report absolute values whose meaning depends on how the master was assembled, so a zero reference has to be captured and stored. If that reference is taken from a frame that failed validation, or if the commanded pose is referenced to the origin of the master's own frame rather than to where the operator's hand actually is, then the first command issued after start-up is a step of the master's entire tip vector.

*Almost every failure in this system is silent.* An arm that has stopped moving looks the same whether it is blocked for a good reason, blocked for a bad one, or receiving no data at all. A large part of the engineering effort went into making internal state legible from outside. One of the more uncomfortable findings of this work is that the measuring instruments failed more often, and more quietly, than the system being measured.

## 1.4 Contributions

Each contribution is stated with its maturity, because the distinction between implemented, verified in simulation, and validated on hardware is load-bearing throughout. **No contribution in this thesis was validated on physical hardware.** [Chapter 8](#) explains why.

1. **A graceful-degradation architecture for a body-worn master.** Per-channel health classification computed over distinct sensor updates, validation scoped to the channels the active mode consumes, run-time channel disabling with published capability loss, and a reduced-degree-of-freedom fallback that keeps position available on a single potentiometer. The cost of each rung is measured on recorded hardware data. *Implemented; measured against 20 430 recorded frames.*
2. **A reduced-degree-of-freedom command mapping** that takes direction from gravity and magnitude from the best-conditioned joints, with an anchor that re-latches on clutch engagement so indexing is unbounded and jump-free. *Implemented; verified in simulation.*
3. **A safety architecture whose blocking is observable.** Every mechanism able to stop the arm registers by name, must state how it clears, and expires into an explicit unknown state if the loop asserting it stops running. *Implemented; fourteen of fourteen injected faults handled.*
4. **A workspace characterisation of the mounted configuration**, including the result that the two arms share no reachable workspace under the as-built mount. This blocks inter-arm handover for geometric reasons that no control or planning change can address. A buildable mount modification that would resolve it is quantified. *Verified in simulation; [Chapter 7](#).*
5. **A verification methodology** requiring an instrument to recover a known answer before a surprising measurement taken with it is reported. *Applied throughout.*

The fifth arose from necessity. Across this project, results that looked like system failures turned out four separate times to be defects in the measuring tool, and validating the instrument first caught errors that would otherwise have entered the record as findings. It is the most transferable outcome of the work.

## 1.5 Thesis structure

[Chapter 2](#) reviews the field by theme and ends with the gap this work addresses. [Chapter 3](#) documents the master arm: its electronics, its firmware and its mechanical build. [Chapter 4](#) records how the control approach reached its present form, including the routes that were tried and abandoned. [Chapter 5](#) describes the system as built. [Chapter 6](#) reports what was measured, labelled throughout with the strength of evidence behind it. [Chapter 7](#) presents the workspace characterisation on its own, because it is a chain of measurements whose consequences determine the shape of the task set. [Chapter 8](#) interprets the results and states the limitations. [Chapter 9](#) sets out what would most improve the system.

# Chapter 2

## Background

This chapter is organised by theme rather than chronology. Five bodies of work bear on the system built here: supernumerary limb design, the operator and wearer split, master interfaces for dissimilar kinematics, shared autonomy, and the measurement of embodiment and trust. The chapter ends by stating what remains open, which is narrower than it first appears.

### 2.1 Supernumerary robotic limbs

The field's founding demonstrations are Parietti and Asada's bracing limbs for aircraft fuselage assembly. Their limbs anchor against the environment to stabilise the wearer's posture, and the same group later extended the approach to whole-body support [22], which turns the wearer's body from a disturbance source into part of a closed chain [21, 20]. That framing matters because it identifies the central mechanical problem early: an arm mounted on a person exerts reaction forces into the person, and the person moves.

Reviews since have mapped the design space along consistent axes [30, 25, 28, 31]. Mounting location trades reach against the moment arm imposed on the wearer. Actuation trades bandwidth against worn mass. Control input ranges from residual body signals to explicit manual command. A more recent quantitative survey adds a useful observation, that the field reports embodiment far more often than it reports task performance, and rarely both [33].

Worn mass is the constraint that binds hardest and is discussed least. Each Gen3 masses 8.2 kg, so the pair with harness exceeds 17 kg. Simulation of the loads such systems impose on the wearer suggests the biomechanical cost is not negligible [18]. [Chapter 8](#) returns to what this implies for session design.

#### PLACEHOLDER: REPRODUCED FIGURE

Parietti and Asada's bracing configuration, showing the limbs anchored against the work surface to close the kinematic chain through the environment. Source: [21], Fig. 1.

*Reproduction requires written permission from the copyright holder. Not yet obtained.*

Figure 2.1: The bracing configuration that established the mechanical framing of the field. To be reproduced from [21] once permission is obtained.

## 2.2 The operator and the wearer as different people

The configuration in this thesis has the arms worn by one person and commanded by another. Fusion is the direct predecessor. It mounted a robotic head and two arms on a worn frame and drove them from a remote operator [26]. Two qualifications must travel with that citation. Fusion’s three modes, Direct, Induced and Enforced, are levels of *physical coupling* between the robot arms and the wearer’s own arms, not levels of machine autonomy; in all three the robot decides nothing. And the publication is a two-page Emerging Technologies demonstration with no user study and no performance numbers. It establishes that the architecture existed. It cannot support any claim about how well it worked.

The evidence about how such systems are experienced comes from elsewhere. Recent work on spatial guidelines for supernumerary limbs in near-body interaction ran a concealed operator, varied the apparent autonomy, and measured the wearer’s physiological arousal and self-reported trust [34]. The finding runs against intuition: higher apparent autonomy made both worse, with skin conductance responses rising and trust ratings falling. Their autonomy was a hidden human and their wearer had no task of their own, so the result does not settle what happens when autonomy is real and the wearer is occupied. It does establish that the wearer’s experience is not a free variable.

Disturbance rejection for the two-person case is partly solved already. Zhang et al. compensate human-induced base motion for a supernumerary arm and report end-effector errors of  $(1.37 \pm 0.58)$  mm with the human shoulder as a floating base [32]. What that work does not cover is the case where the disturbance originates in a nervous system the operator has no efference copy of. When wearer and operator are the same person, the base motion is partly predictable to the controller because the person commanding it also caused it. When they are different people, it is not.

## 2.3 Master interfaces for dissimilar kinematics

Where master and slave differ in scale or morphology, the mapping must be chosen rather than assumed. Guggenheim and Asada argue for miniature physical masters on the grounds that geometric similarity supplies feedback inherently, without a separate haptic display [7]. The argument is persuasive for similarity of *form*. It says nothing about similarity of *scale*, and scale is where the present system departs: a ratio of roughly three to one between master and slave reach means identical joint angles produce very different tip displacements.

Two mappings are available in principle. Joint-space correspondence sends master joint angles to slave joint angles directly. It preserves the arm’s posture, which matters when the posture itself must avoid something, and it requires every joint of the master to be measured accurately. Task-space correspondence maps tip displacement to end-effector displacement, decoupling the two chains at the cost of leaving the slave’s redundant degree of freedom unspecified. [Chapter 4](#) records why this project moved from the first to the second.

Supplementary feedback is the other axis. Pinardi et al. review what additional sensory channels do for control and embodiment in movement augmentation, and find the two do not move together [23]. Feedback that improves control does not reliably improve the sense of ownership, which is a caution against treating either as a proxy for the other.

## 2.4 Shared autonomy

The shared-autonomy literature addresses a different question from the teleoperation literature: not how to transmit the operator’s command, but how much of it to obey. Dragan and Srinivasa formalise the arbitration as policy blending, in which a predicted user goal and the user’s actual input are combined with a weight that depends on the confidence of the prediction [5]. The framework is explicit that blending too aggressively on a wrong prediction is worse than not blending at all, which is the property that governs the arbiter built here.

Javdani et al. recast the problem as a partially observable Markov decision process over the user’s goal and approximate the intractable solution with hindsight optimisation [13]. Their user study found that assistance which does not wait for confidence to be high lets operators finish faster with less input. The lesson taken here is narrower than the method: an assistance policy needs an explicit representation of its own uncertainty, and needs a defined behaviour for the case where the distribution over goals is flat.

Neither line of work was developed for a robot mounted on a second person. In both, the cost of assisting toward the wrong goal is a slower trial. Here it is a manipulator moving toward someone who did not command it.

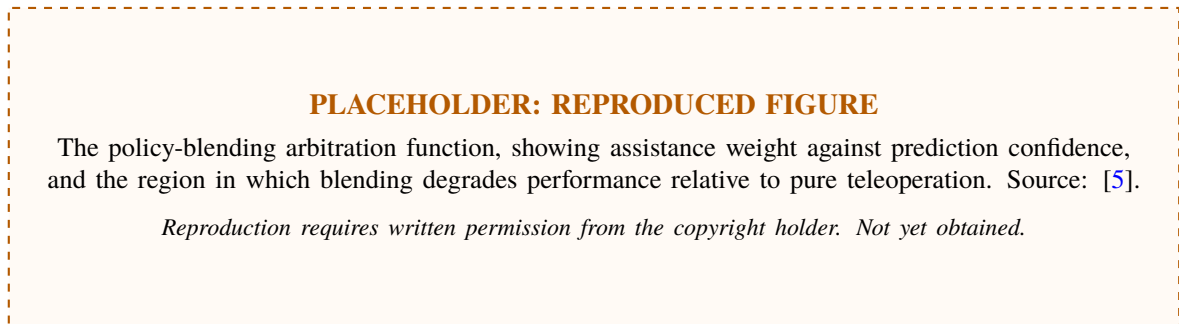


Figure 2.2: Arbitration as a function of prediction confidence. To be reproduced from [5] once permission is obtained.

## 2.5 Measuring embodiment, workload and trust

Body ownership over an artificial limb is measurable, and the instruments are long established. The rubber hand illusion showed that ownership can be induced over an object that is plainly not one’s hand [3], and Longo et al. decomposed the resulting experience psychometrically into components including ownership, agency and location [16]. The decomposition matters for a supernumerary limb because ownership and agency can move independently: an operator may feel authorship of a motion without feeling the arm is part of them, and a wearer may feel neither while still bearing the consequences.

Workload is conventionally measured with the NASA Task Load Index [8], which separates mental, physical and temporal demand from effort, performance and frustration. For a two-person system the index has to be administered to both people, and the two will not load the same way. Physical exertion is better tracked with a Borg category-ratio scale [2], which is the appropriate instrument for the operator’s uncompensated arm and for the wearer’s carried mass.

Interface design has its own standard. ANSI/ISA-101.01 sets out a philosophy for operator displays in process automation [10], and its central principle transfers directly to a robot console: colour is a scarce alarm channel, so a normal state should be largely uncoloured rather than green. A display that paints five items red simultaneously

has communicated the same amount as one that paints none.

## 2.6 The gap

The gap is narrower than a first reading suggests, and stating it precisely matters more than claiming it is wide.

Fusion established the operator and wearer architecture but measured nothing [26]. Shared autonomy has been shown to raise operator performance, in settings where the robot is not attached to anybody [5, 13]. Work on near-body interaction has shown that increasing apparent autonomy lowers the wearer’s trust and raises their physiological arousal [34], in a setting where the autonomy was simulated by a concealed human and the wearer had no task.

So it is not true that nobody has looked at any of this. What has not been done is to measure both ends in one system: the operator’s performance and the wearer’s experience, under the same autonomy, in the same trial. Until that is done the exchange rate is unknown. If shared autonomy buys twenty per cent in completion time and costs a point of wearer trust, that may be a good trade or a bad one, and there is currently no basis for saying which.

The engineering contribution of this thesis is a system in which that question could be asked. The measurement itself is not made here, and [Chapter 8](#) sets out what stands between the system as built and a session with two people in it.

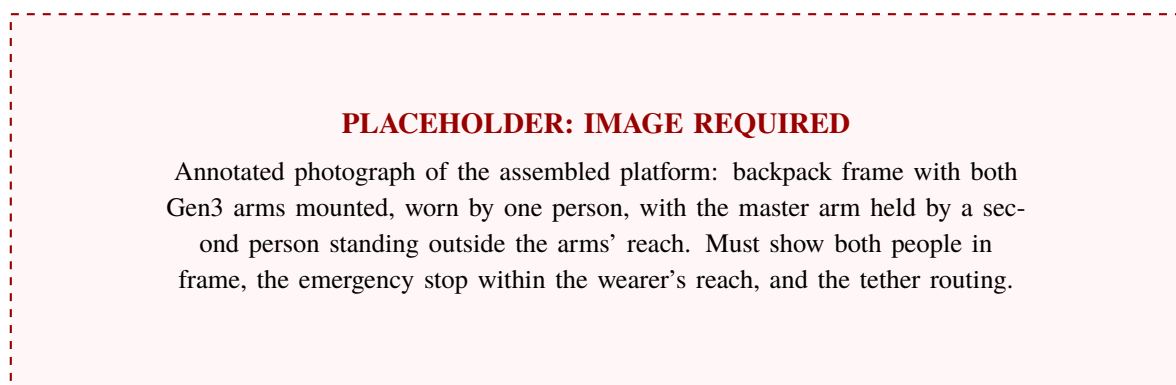


Figure 2.3: The configuration this thesis addresses. Photograph required before submission.

# Chapter 3

## The master arm

The master is a miniature mannequin reproducing the Gen3's joint sequence at roughly a third of its scale. The operator moves it with the fingers of one or both hands. This chapter documents it completely: the mechanical chain, the electronics, the firmware that turns fourteen analog channels and two inertial sensors into a serial frame, and the signal conditioning applied on the way. Source is at `firmware/master_arm/teensy_final.ino`.

### 3.1 Mechanical chain

Each arm carries seven potentiometer-instrumented joints in an alternating pattern: a roll about the preceding link's own axis, then a bend perpendicular to it, repeated. [Figure 3.1](#) gives the sequence with the measured link lengths, which sum to 0.272 m.

The alternation is not incidental to the sensing design. A roll joint moves the tip only through a downstream bend, because rotating a straight arm about its own axis translates the tip by zero. A bend moves the tip directly. That asymmetry is why the degradation ladder in [Chapter 6](#) is ordered the way it is, and why the loss of J2 or J4 costs reach while the loss of J3 or J5 costs almost nothing.

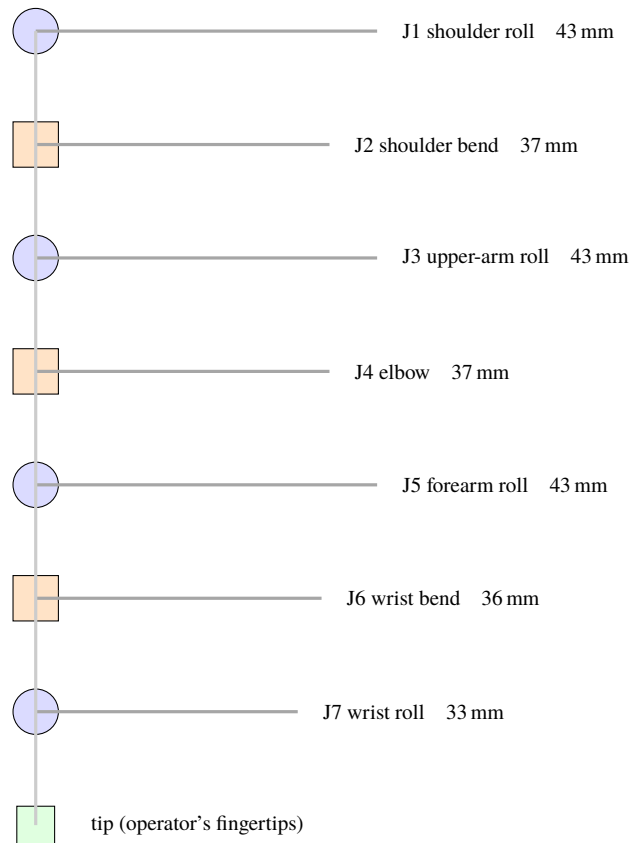
### 3.2 Electronics

A Teensy 4.1 reads every sensor and emits one text frame per cycle. The sensor complement is fourteen rotary potentiometers, two MPU-6050 inertial measurement units, two force-sensitive resistors and two momentary buttons. [Figure 3.4](#) gives the block diagram and the pin assignment; [Table 3.1](#) gives the assignment in tabular form.

Table 3.1: Pin assignment. Sixteen analog channels are required and sixteen are available once the two occupied by the default I<sup>2</sup>C bus are excluded.

| Function                  | Count | Pins                       | Note                            |
|---------------------------|-------|----------------------------|---------------------------------|
| K1 (left) potentiometers  | 7     | A0 A1 A2 A3 A6 A7 A8       | no I <sup>2</sup> C pins        |
| K2 (right) potentiometers | 7     | A9 A10 A11 A12 A13 A16 A17 | no I <sup>2</sup> C pins        |
| Force-sensitive resistors | 2     | A14, A15                   | left, right                     |
| Buttons                   | 2     | digital 2, 3               | INPUT_PULLUP                    |
| I <sup>2</sup> C (Wire)   | 2     | pin 18 = A4, pin 19 = A5   | <b>excluded from analog use</b> |

Analog-capable pins on Teensy 4.1: 18. Reserved for I<sup>2</sup>C: 2. Remaining: **16**. Required: 14 + 2 = **16**.



**Total extension**  $\sum L_i = 0.272$  m, against the Gen3's 0.902 m reach. The ratio is 3.3:1, so no joint-to-joint mapping is available and the correspondence has to be made in task space.

○ roll about the preceding link's axis      □ bend perpendicular to it.

The alternation matters for degradation: a *roll* moves the tip only through a downstream bend, so losing one costs little radial information, whereas losing a *bend* (J2 or J4) removes reach outright.

Figure 3.1: The master's seven joints with measured link lengths, drawn at all joints zero so the lengths read off directly.

### PLACEHOLDER: IMAGE REQUIRED

CAD render of one master arm, exploded, showing the seven potentiometer housings, the shaft couplings at each joint, the cable routing channel along each link, and the wrist platform carrying the MPU-6050 and the force-sensitive resistor. Must be dimensioned and must show how the cable bundle crosses each rotating joint, since that crossing is where the channel failures discussed in [Chapter 6](#) originate.

Figure 3.2: Master arm assembly. CAD render required before submission.

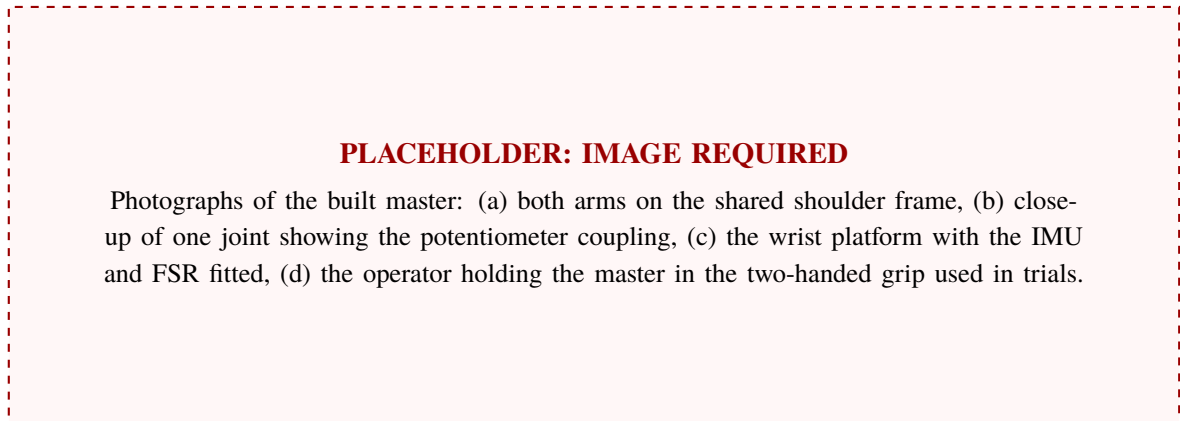


Figure 3.3: The assembled master. Build photographs required before submission.

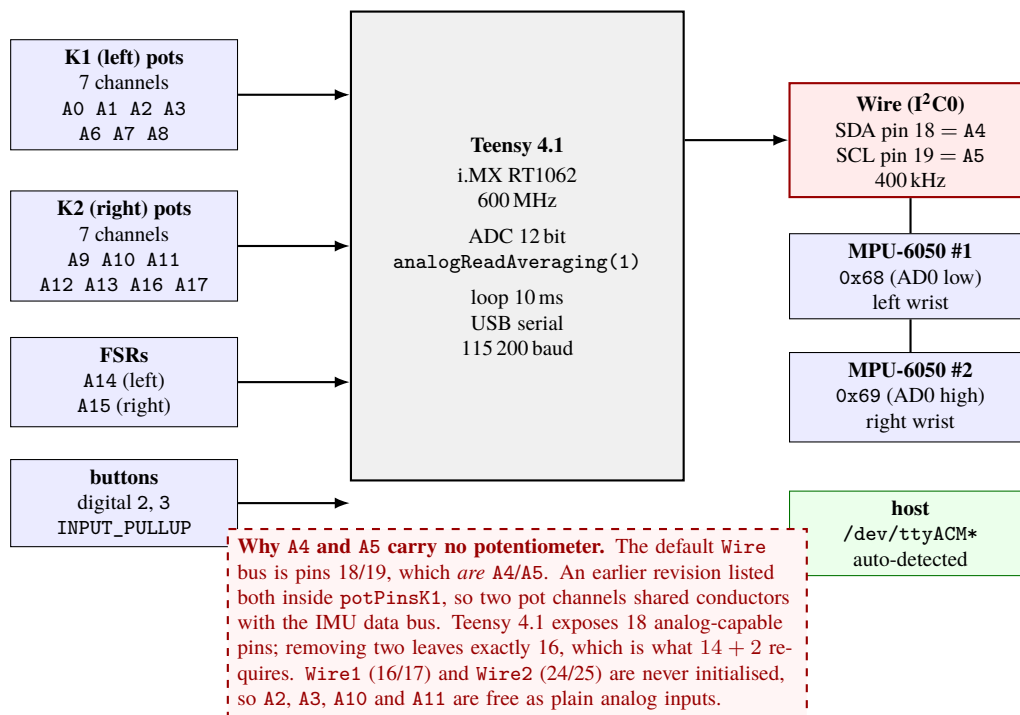


Figure 3.4: Electronics architecture and pin assignment. The call-out explains why two analog-capable pins carry no potentiometer.

### 3.2.1 The A4 and A5 collision

An earlier revision of the firmware listed A4 and A5 inside the left arm's potentiometer array. Those two pins are not free. On the Teensy 4.1 the default Wire bus, which both inertial measurement units use, is physically pins 18 and 19, and those pins *are* A4 and A5. Two potentiometer channels were therefore sharing conductors with the inertial sensors' data bus.

The failure mode this produces is not a clean one. Calling `analogRead()` on a pin that the I<sup>2</sup>C peripheral is driving reconfigures that pin, so the effect appears as intermittent bus corruption on the inertial sensors and as meaningless values on two potentiometer channels, with the severity depending on the phase relationship between the analog read and the bus transaction. Nothing about it looks like a pin conflict from the outside.

The resolution was to exclude both pins from analog use entirely. The arithmetic then works exactly: Teensy 4.1 exposes eighteen analog-capable pins, removing two leaves sixteen, and sixteen is what fourteen potentiometers plus two force sensors require. There is no spare, which is worth recording, because any future channel needs either a multiplexer or a second board.

The two secondary I<sup>2</sup>C buses are safe to use as plain analog inputs and the design does so [24]. Wire1 occupies pins 16 and 17 (A2, A3) and Wire2 occupies pins 24 and 25 (A10, A11), but neither peripheral is ever initialised: the firmware calls only Wire. A bus that is never begun does not claim its pins, so all four remain ordinary analog inputs and all four carry potentiometers.

That headroom is the obvious route to a third and fourth inertial sensor. The MPU-6050 family offers exactly two addresses, 0x68 and 0x69, selected by the AD0 line, and both are already taken by the two wrist sensors. A third sensor cannot join the same bus. Bringing up Wire1 on pins 16 and 17 would carry another pair, at the cost of the two potentiometer channels currently on A2 and A3. Chapter 6 quantifies why that trade may be worth making.

## 3.3 The inertial measurement driver

The firmware includes a local MPU6050.h rather than an external library. The reason is build portability: a sketch that depends only on Wire compiles on any Arduino-compatible toolchain without a library manager resolving versions, and the driver's behaviour is then fixed by the source in the repository rather than by whatever version happens to be installed.

**CONFIRM:** MPU6050.h was not among the firmware files handed over and is not in the repository. The interface below is the one the sketch calls and is therefore certain; the register sequence is as specified by the author and matches the device documentation, but the file itself should be attached to the appendix before submission.

Three entry points are used. `initialize()` wakes the device, `testConnection()` confirms it is present and addressed correctly, and `getMotion6()` returns six signed 16-bit values.

**Waking the device.** The MPU-6050 powers up in sleep, with the SLEEP bit of PWR\_MGMT\_1 (register 0x6B) set. Until that bit is cleared the sensor registers read as zero [12]. Clearing it is the whole of initialisation for this application, since the power-on defaults for range and filtering are the ones wanted.

**Confirming presence.** WHO\_AM\_I (register 0x75) returns a fixed identity value. Reading it distinguishes three states that otherwise look identical from the outside: a sensor that is present and responding, a sensor that is absent

so the bus read returns nothing, and a sensor that is present but strapped to the other address. The firmware prints the verdict per sensor at start-up, and that line is the first thing to check when one arm reports no orientation.

**Reading the measurement.** All six axes plus temperature occupy fourteen consecutive registers beginning at ACCEL\_XOUT\_H (0x3B). The driver reads all fourteen in a single burst. This is not only a bandwidth optimisation. The device latches its output registers for the duration of a burst read, so a single transaction returns six axes from one sampling instant, whereas six separate reads can straddle an internal update and return a mixture of two. For an attitude estimate built from the ratio between axes, a mixed sample is a wrong attitude rather than a noisy one.

Each axis arrives high byte first and is reassembled as a signed 16-bit value. The two bytes of temperature sit between the accelerometer and gyroscope blocks and are discarded here.

### 3.3.1 Scale factors

At the power-on defaults the accelerometer is at  $\pm 2$  g full scale and the gyroscope at  $\pm 250^\circ/\text{s}$ , giving sensitivities of 16 384 LSB/g and 131 LSB/ $^\circ/\text{s}$  [11]. The firmware never changes the range registers, so these are the values that apply.

Table 3.2: Inertial sensor scaling at power-on defaults.

| Quantity     | Full scale               | Sensitivity                | Resolution                                  |
|--------------|--------------------------|----------------------------|---------------------------------------------|
| Acceleration | $\pm 2$ g                | 16 384 LSB/g               | 61 $\mu\text{g}$ per LSB                    |
| Angular rate | $\pm 250^\circ/\text{s}$ | 131 LSB/ $^\circ/\text{s}$ | $7.6 \times 10^{-3}^\circ/\text{s}$ per LSB |

A value of 8192 LSB/g is widely copied in example code and is wrong for this configuration. It corresponds to the  $\pm 4$  g range, which the device only enters if ACCEL\_CONFIG is written. Used against the default range it halves every reported acceleration. That error does not announce itself in the elevation estimate, because elevation is computed from the direction of the acceleration vector and a uniform scale error cancels in a normalised vector. It announces itself in the accelerometer gate, which compares the vector’s magnitude against 1 g to decide whether the sensor is measuring gravity alone: with the wrong scale, a stationary sensor reports 0.5 g and the gate rejects every frame. The failure appears as a master that never produces a valid orientation, which looks like a dead sensor.

## 3.4 Signal conditioning

### 3.4.1 Potentiometers

Each channel is read at 12-bit resolution with hardware averaging disabled, then passed through an exponential moving average implemented in integer arithmetic:

$$e_{n+1} = \frac{\alpha r_n + (256 - \alpha) e_n}{256}, \quad \alpha = 64, \quad (3.1)$$

which is a smoothing factor of 0.25 per sample, realised as a shift rather than a division. Integer arithmetic was chosen because the loop runs at 100 Hz across sixteen channels and the Teensy’s floating-point unit is not needed for a filter whose input is a 12-bit integer.

The filtered count maps to degrees by a clamped linear transform:

$$\theta = 360^\circ \cdot \text{clamp}\left(\frac{a - 50}{4045 - 50}, 0, 1\right). \quad (3.2)$$

The clamp limits are the substantive part. A potentiometer wiper never reaches the extreme of its track, and the ends of the ADC range are where the reading is least linear, so the transform discards the outer 50 counts at the bottom and 50 at the top. The remaining 3995 counts span  $360^\circ$ , giving  $0.090^\circ$  per count.

### 3.4.2 A start-up transient in the filter initialisation

Equation (3.1) operates in raw ADC units: the steady state of the recurrence for a constant input  $r$  is  $e = r$ . The initialisation in `setup()` does not match. It seeds each accumulator with `analogRead(pin) << 8`, which is 256 times the raw reading, a value consistent with a fixed-point formulation the filter does not use.

Simulating the recurrence from that seed gives the transient in Table 3.3. Every channel reads a railed  $360^\circ$  for roughly the first 190 ms after reset and takes 440 ms to settle.

Table 3.3: Filter output after reset, simulated from Eq. (3.1) with a constant input of 2000 counts and the seed used in `setup()`.

| Cycle | Accumulator | Reported angle          |
|-------|-------------|-------------------------|
| 1     | 384 500     | $360.0^\circ$ (railed)  |
| 10    | 30 719      | $360.0^\circ$ (railed)  |
| 20    | 3616        | $321.3^\circ$           |
| 25    | 2382        | $210.1^\circ$           |
| 30    | 2090        | $183.8^\circ$           |
| 44    | 2000        | $180.0^\circ$ (settled) |

Two consequences follow, and the second is the one that has cost time. Any calibration capture taken within half a second of a board reset records a railed or settling value rather than a joint angle. And re-opening the serial port resets the board, so a calibration script that opens the port and immediately samples is guaranteed to capture the transient. This is the same window in which the inertial sensors are also settling, and the effects are indistinguishable at the ROS level: both appear as a channel reporting a plausible but wrong constant.

The existing calibration procedure already requires a settled stream before sampling (Fig. 5.9), for reasons that were diagnosed from the gyroscope alone. The analysis here shows the potentiometers need the same protection for a different reason. Correcting the seed to `analogRead(pin)` removes the transient entirely and changes nothing else, since the steady-state behaviour of Eq. (3.1) is unaffected.

**MEASURE:** confirm the corrected seed on hardware and re-time the settling window. The simulation above is arithmetic on the shipped recurrence, not a measurement of the board.

### 3.4.3 A candidate explanation for the observed update rate

The frame is emitted at 100 Hz, but recorded captures show each potentiometer producing a new *distinct* value at only 14.7 to 17 Hz, so 82 to 88 % of consecutive rows repeat. That aliasing invalidated every per-row statistic computed before it was noticed (Chapter 6).

The firmware offers a mechanism. Angles are printed with one decimal place, and  $0.1^\circ$  is 1.11 ADC counts. A smoothing factor of 0.25 moves the filter only a quarter of the way to a new input in one cycle, so a slow joint

motion produces sub-count filter steps and the printed value changes only when the accumulated motion crosses a printing boundary. The effect is quantisation in the output format compounded with the filter, not a slow sensor.

**MEASURE:** this explanation is consistent with the firmware but has not been isolated. It is directly testable with `scripts/virtual_teeny.py`, which streams frames in the same wire format with known ground truth: sweep a channel at a known rate and count distinct printed values against the predicted boundary crossings. If confirmed, printing two decimals or raising  $\alpha$  recovers the full rate.

### 3.4.4 Force-sensitive resistors

Both force sensors are read on the same 12-bit ADC as the potentiometers and transmitted as integers. Measured pad values are 6 to 191 counts unsqueezed and 3603 (left) and 3842 (right) at full squeeze, with cross-talk of 3.7 % and 0.4 % under load only, so no mixing matrix is required.

**CONFIRM:** the intended design applies a floating-point exponential average to the force channels, so they carry decimal precision rather than integer counts, and the ROS-side parser accepts a decimal in both force fields. The firmware handed over transmits %d and applies no filter to these two channels. Either the float version was not the one delivered or the change was not made; this should be resolved before the appendix listing is fixed. The gripper mapping in [Chapter 5](#) does not depend on it, since it applies its own deadband and hysteresis downstream.

### 3.4.5 Buttons

Both buttons are wired as two-wire switches to ground with `INPUT_PULLUP`, so the pin reads high when open and low when pressed and no external resistor is needed. The firmware converts each press into a latching toggle: a transition is accepted only if the raw level differs from the last accepted level and at least 40 ms have passed since the last accepted transition, and the output state flips only on the falling edge.

Debouncing at the source matters more here than it usually would, because the button is the clutch. A bounce that registers as two transitions toggles the clutch twice, which disengages and re-engages it within a few milliseconds and re-latches the anchor at whatever pose the master happened to be in. The downstream clutch logic adds its own debounce and a quasi-static gate for the same reason.

## 3.5 The serial frame

One frame is emitted per cycle, at 100 Hz, as ASCII text terminated by a newline. [Table 3.4](#) gives the fields in order. The format is deliberately human-readable: a frame can be read directly from a terminal to tell a wiring fault from a firmware fault without any host software running.

Three properties of this format are relied on downstream. Inertial values are transmitted *raw*, in LSB, so the scale factors are applied on the host and can be corrected without reflashing. The token `k1j1` at the start of every frame is what the host's port auto-detection sniffs for, which is how the board is found when it moves between `/dev/ttyACM0` and `/dev/ttyACM1` across re-attachments. And the buttons carry a latched state rather than an instantaneous level, so a host that misses frames cannot miss a press.

Table 3.4: Serial frame, in transmission order. Fields are comma-separated; the frame ends with a newline. Buffer is 460 bytes against a typical frame of roughly 285.

| Field       | Count | Format | Meaning                                  |
|-------------|-------|--------|------------------------------------------|
| k1j1...k1j7 | 7     | %.1f   | left arm joint angles, 0 to 360°         |
| k2j1...k2j7 | 7     | %.1f   | right arm joint angles, 0 to 360°        |
| K1IMU       | 6     | %d × 6 | left $a_x a_y a_z g_x g_y g_z$ , raw LSB |
| K2IMU       | 6     | %d × 6 | right, same order                        |
| fsr1, fsr2  | 2     | %d     | grip force, raw counts                   |
| btn1, btn2  | 2     | %d     | latched toggle state, 0 or 1             |

### 3.6 Summary

The master supplies fourteen joint angles, two three-axis accelerations, two three-axis angular rates, two grip forces and two latched buttons, at 100 Hz over USB serial. Of that, the position pipeline in [Chapter 5](#) consumes three or four potentiometer channels per arm and one acceleration vector. The remainder is either used for orientation, used for the gripper and clutch, or held in reserve against the loss of a channel that is being used.

# Chapter 4

## How the control approach reached its present form

The system described in [Chapter 5](#) is not the one the planning report proposed, and the route between them is part of the result. This chapter records the sequence, the alternatives that were considered at each branch, and what each abandoned approach contributed to the one that shipped. The divergence from the planning report is analysed separately in `thesis_report/DIVERGENCE.md`.

CONFIRM: dates and durations of the simulation phase with Dr. Zhang. The sequence below is accurate; the calendar is reconstructed from repository history and supervision notes rather than from a contemporaneous log.

### 4.1 Phase 1: joint-space control in simulation

The planning report proposed validating the master and slave mapping in Isaac Lab or MuJoCo before touching hardware. MuJoCo [27] was chosen over Isaac Lab [19]. It starts in under a second, its model format is a single readable file, and for a kinematic mapping question its contact solver is irrelevant, so the heavier alternative bought nothing.

The first mapping was the obvious one. The master reproduces the Gen3's joint sequence, so each master potentiometer angle was sent to the corresponding slave joint after zero-referencing. Joint one to joint one, and so on through seven.

Three things were learned, and all three survived into the delivered system.

**The mapping is dominated by convention handling, not by control.** Almost all of the difficulty was in what a joint angle *means*: which direction is positive, where zero sits, whether the value wraps, and over what interval. This is where the degree and radian conversion described in [Chapter 1](#) was first written, and it is the reason it later became an isolated, unit-tested module rather than an inline expression. The lesson transferred directly: the arithmetic that converts between conventions deserves more test coverage than the control law it feeds.

**An unconstrained joint mapping will drive the arm into things.** In simulation this is a visual annoyance. On the delivered platform, where the arm is mounted beside a person's head, it is the entire safety problem. A joint-space mapping has no representation of the wearer, so nothing in the formulation can express the constraint that matters most. That observation is what motivated the move to a framework with a collision model in it.

**Sensor faults are the dominant failure mode, not control faults.** Injecting a stuck or noisy channel showed the arm following it faithfully, because a direct joint mapping has no notion of a reading being implausible. The channel-health architecture in [Chapter 5](#) is a direct response.

#### 4.1.1 Why joint-space control was abandoned

Joint-space correspondence has a genuine advantage that should be recorded before the case against it: it fixes the redundant degree of freedom for free. The Gen3 has seven joints for a six-dimensional task, so a task-space command leaves one dimension unspecified and something has to choose it. Under a joint mapping the operator chooses it implicitly, by the posture they put the master in.

It was abandoned for two reasons that are both about the master rather than the method.

The first is scale. The master's links sum to 0.272 m against the Gen3's 0.902 m reach. Equal joint angles therefore produce tip displacements differing by a factor of about three, so the operator's motion and the robot's motion are related by a gain that varies with posture. Working in task space makes that gain an explicit, tunable scalar instead.

The second is decisive. A joint mapping requires *every* joint to be measured, because each one contributes directly to the commanded configuration. It has no graceful degradation at all: a single failed channel sends a wrong angle to the corresponding slave joint, and there is no reconstruction available because nothing else in the mapping observes that joint. Given the argument in [Chapter 1](#) that a body-worn master will lose channels, a formulation that cannot survive losing one was not viable.

## 4.2 Phase 2: the move to ROS 2

MuJoCo answers questions about dynamics. It does not supply a collision-aware inverse kinematics service, a controller stack, or drivers for the actual arms, and by the end of phase 1 all three were needed. The system moved to ROS 2 Jazzy [\[17\]](#).

Four capabilities came with the move, and each closed a problem left open by phase 1.

- **A collision model containing the wearer.** MoveIt checks candidate configurations against a scene, so the wearer's body can be part of the model. This is the capability a joint mapping could not express at all.
- **Inverse kinematics with redundancy resolution.** TRAC-IK solves the seven-joint problem with random restarts and a distance objective [\[1\]](#), which makes task-space commanding practical and lets the redundant degree of freedom be re-seeded on failure.
- **A controller stack with a hardware abstraction.** The same trajectory interface addresses a mock system and the real driver, so everything upstream is developed and tested unchanged and the substitution is one launch argument.
- **Vendor drivers.** The manufacturer supplies a ROS 2 driver, so the alternative was writing one.

The cost was real. ROS 2 introduced a discovery layer, a middleware configuration, and a process lifecycle, and a substantial fraction of the faults recorded in this project are in that layer rather than in the robot. [Chapter 6](#) reports them, because they are the ones a successor will meet first.

### 4.2.1 What was kept from simulation

Simulation did not stop being useful when hardware became available. It became the place where anything that could not be attempted on a person is attempted. Every workspace measurement in [Chapter 7](#), every fault injection, and every clearance figure is simulated, and that is a deliberate allocation rather than a fallback. The mock hardware interface is what makes it possible to exercise the entire stack, including the trajectory controllers and the emergency stop, without an arm attached.

The limit of that substitution is stated where it binds. The mock returns identically zero for every force, so anything derived from a force reading is structurally undefined rather than merely unmeasured.

## 4.3 Phase 3: from Cartesian control to a reduced-degree-of-freedom command

Task-space control was not the end of the sequence. The first Cartesian implementation computed the master's tip pose by forward kinematics from all seven potentiometer angles and commanded the end effector to the scaled displacement. That is the textbook formulation and it failed on this hardware.

The failure was directional rather than noisy. Decomposing the commanded positions from five distinct gestures put 93.4 % of the variance on a single axis, and placed the pose commanded by raising the master *below* the pose commanded by lowering it. A mapping that inverts up and down is not a mapping with a gain error.

The cause is that forward kinematics compounds every joint angle in the chain. An error in joint three propagates through joints four to seven, so the tip direction inherits the sum of seven absolute-angle calibration errors, and the inertial mount calibration had already shown those errors to be large enough to be physically impossible in places.

The response was to stop computing direction from the chain. Position is now built in spherical form about the master's shoulder, taking each component from whichever sensor observes it most directly: elevation from the gravity vector, azimuth from the single shoulder-roll potentiometer, and radius from the forward-kinematic magnitude, which depends only on the bend joints and is insensitive to roll errors. [Chapter 5](#) gives the formulation and [Chapter 6](#) the measured consequences, including the one axis that this does not fix.

### 4.3.1 Alternatives considered at this branch

**Recalibrate the potentiometers and keep full forward kinematics.** Rejected on cost and on durability. It requires an external angular reference per joint, and it has to be redone whenever a joint is reassembled. It also fixes nothing about channel loss, since forward kinematics still needs every joint.

**Take azimuth from forward kinematics instead of from joint one.** Built and measured, then rejected. It made direction worse on both arms, moving the mean reversal error from 12.6° to 33.3° on the left arm and from 26.2° to 39.5° on the right. It fails for the same reason the full chain fails: it compounds four suspect angles where the single-joint estimate compounds one.

**Integrate the gyroscope for azimuth.** Implemented, measured, and left disabled by default. It observes rotation about vertical directly and so avoids both error sources, and it halves the residual on the left arm. It is not enabled because one of the two sensors drifts at 44 °/min while stationary, and because the capture that would validate it needs a moving master and does not yet exist.

**Add a second inertial sensor on the upper arm.** Not attempted, for the hardware reason in [Chapter 3](#): both

available addresses are occupied. It remains the most promising route, and [Chapter 6](#) quantifies what it would buy.

## 4.4 Summary of the sequence

Table 4.1: Development sequence, with the reason each stage ended.

| Stage                  | Mapping                             | Why it ended                                                                                                                                |
|------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| MuJoCo, phase 1        | joint to joint                      | No collision model, so the wearer could not be represented. No degradation path: one failed channel corrupts one slave joint irrecoverably. |
| ROS 2, first Cartesian | tip pose by full forward kinematics | Direction collapsed. 93.4 % of variance on one axis, and raising the master lowered the arm. Seven compounded calibration errors.           |
| ROS 2, delivered       | spherical, per-component sensing    | Current. Elevation and reach are correct; azimuth remains limited by the lever-arm coupling described in <a href="#">Chapter 6</a> .        |

The through-line is that each stage failed because of what it assumed about the *sensors*, not because of what it assumed about the control. Joint mapping assumed every channel works. Full forward kinematics assumed every angle is accurate. The delivered mapping assumes only that gravity points down and that the best-conditioned joints are identifiable at run time, which are assumptions the hardware can actually support.

# Chapter 5

## System as built

This chapter describes the delivered system: how it is decomposed into packages, how a master pose becomes an arm command, what happens when sensing degrades, and what can stop the arm. [Chapter 3](#) covered the master hardware and firmware; this chapter starts where the serial frame arrives.

### 5.1 Architecture

The workspace is six packages with a one-way dependency graph ([Fig. 5.1](#)). The direction is a design constraint rather than a convention. Direct teleoperation is the baseline condition of every planned experiment, so if the teleoperation package imported anything from the autonomy package the comparison would be circular. This was verified two ways: no such import exists anywhere in the source, and the inverse-kinematics follower still reaches its running state with the autonomy and perception packages physically deleted from the install tree.

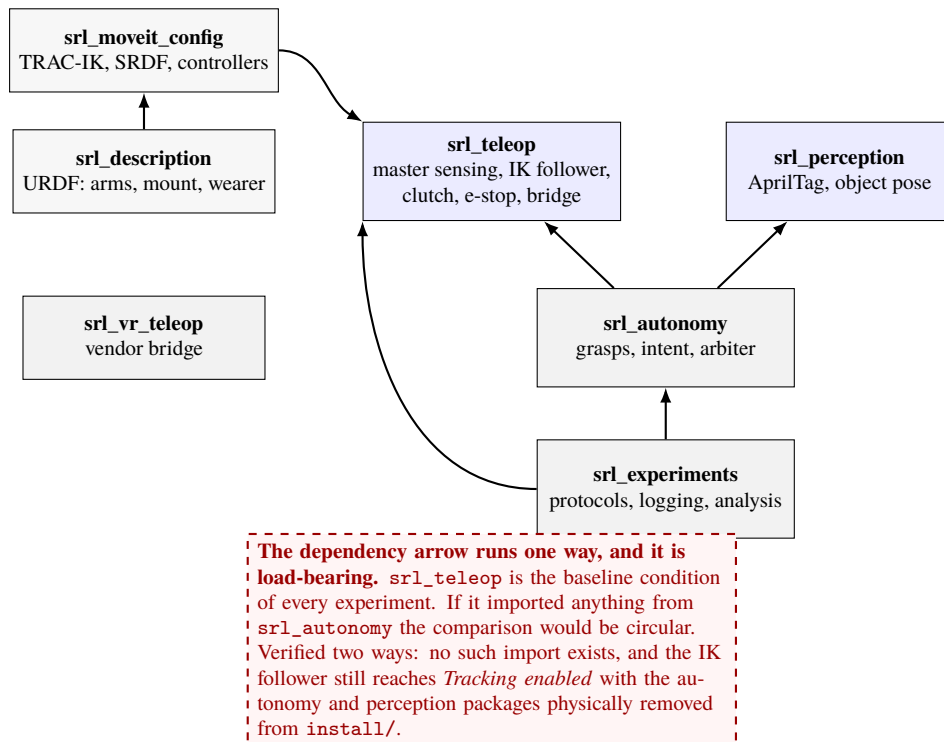


Figure 5.1: Package decomposition and the one-way dependency graph.

## 5.2 From master pose to arm command

### 5.2.1 Spherical decomposition

The commanded position is built in spherical form about the master's shoulder rather than by forward kinematics from seven joint angles. [Chapter 4](#) gives the measurements that forced the change. Each component comes from whichever sensor observes it most directly:

$$\phi = \arcsin\left(\hat{a} \cdot \frac{\mathbf{g}}{\|\mathbf{g}\|}\right) \quad \text{elevation, from the gravity vector} \quad (5.1)$$

$$\psi = \theta_1 - \theta_1^0 \quad \text{azimuth, from the shoulder roll} \quad (5.2)$$

$$r = \|\text{FK}(\theta_1, \theta_2, \theta_3, \theta_4, 0, 0, 0)\| \quad \text{radius, magnitude only} \quad (5.3)$$

Here  $\hat{a}$  is the arm's long axis expressed in the sensor's own frame, captured once with the master hanging at rest, and  $\mathbf{g}$  is the measured acceleration.

Three properties follow, and each addresses a specific failure of the previous formulation. [Equation \(5.1\)](#) is mount-independent, because  $\hat{a}$  is captured through the same mounting it is later used with, and drift-free, because it integrates nothing. [Equation \(5.2\)](#) depends on one directly measured zero-referenced quantity rather than on four compounded ones. [Equation \(5.3\)](#) takes a magnitude, which is dominated by the bend joints and insensitive to errors in the rolls, so a dead roll channel can be passed through as zero without corrupting it.

The sign convention in [Eq. \(5.1\)](#) is asserted at capture time: hanging is  $-90^\circ$ , horizontal is  $0^\circ$ , vertical is  $90^\circ$ . The specification originally called for a negation in both  $\hat{a}$  and the elevation expression, which compound and place the resting pose at  $90^\circ$ , inverting the vertical axis alone. Only one negation is correct.

### 5.2.2 Anchoring, scaling and indexing

The commanded end-effector position is

$$\mathbf{p} = \mathbf{p}_{\text{anchor}} + s(\mathbf{d} - \mathbf{d}_{\text{ref}}), \quad (5.4)$$

with  $\mathbf{d}$  the master tip in the robot frame,  $\mathbf{d}_{\text{ref}}$  the reference latched at clutch engagement, and  $s$  a per-arm scale.

The reference is latched from the first *valid* frame after boot, at the master's actual pose. An earlier version initialised it to the origin of the master's frame, which made the first command  $\mathbf{p}_{\text{anchor}} + s\mathbf{d}$ : a step of the master's entire tip vector, roughly 0.22 m, in whatever direction the master happened to be lying. The only master pose that did not produce a jump was  $\mathbf{d} = 0$ , which is why the procedure previously required the operator to hold a calibrated neutral before start-up. Requiring validity is not incidental. Latching from a frame that failed validation offsets every subsequent command.

Scale is re-read every cycle so it can be retuned live, and a change re-bases the anchor so the commanded pose stays continuous:

$$\mathbf{p}_{\text{anchor}} += (s_{\text{old}} - s_{\text{new}})(\mathbf{d} - \mathbf{d}_{\text{ref}}). \quad (5.5)$$

Without [Eq. \(5.5\)](#) a scale change displaces the arm by  $(s_{\text{new}} - s_{\text{old}})$  times the current displacement, which is

largest exactly when the operator is furthest from the engagement point.

Releasing the clutch freezes the command; re-engaging re-latches  $\mathbf{d}_{\text{ref}}$  at the master's new pose. Successive cycles therefore accumulate, and the reachable set is bounded by the robot rather than by the master's range. Engagement is gated on the master being quasi-static, because latching a reference while the master is moving captures a reference that is already stale.

## 5.3 Graceful degradation

### 5.3.1 Channel health

Each channel carries a verdict computed from its own recent history, over *distinct* sensor updates rather than over transmitted rows. The distinction is not pedantic: at the frame rate roughly 82 to 88 % of consecutive rows repeat, so any statistic computed across rows is dominated by duplicates. Every channel statistic computed that way returned a median absolute difference of exactly zero, which was read for a long time as evidence of dead channels.

Four verdicts are assigned, in order:

**Dead.** Circular range below  $5^\circ$ , or dropouts above 90 %.

**Incoherent.** More than 5 % of updates jump by more than  $60^\circ$ , which at the observed update rate is roughly  $2500^\circ/\text{s}$ .

**Intermittent.** Dropouts above 2 %.

**Alive.** Coherent, range at least  $20^\circ$ , dropouts at most 2 %.

The incoherent verdict is the one that did not exist in earlier versions and is the one that matters. A failing potentiometer does not go quiet. It returns values spanning a wide interval which are not a trajectory, with consecutive updates unrelated to one another. Range alone calls that healthy, which is how a broken channel keeps being trusted.

Range must be computed as the smallest arc containing every sample. Unwrapping and subtracting is invalid for a rotary sensor, because across a dropout the unwrap cannot know how far the joint moved and the accumulated offset runs away; that method reported  $314\,363^\circ$  on a  $360^\circ$  sensor. Samples outside  $[0, 360]$  are rejected first, since a small fraction are serial parse glitches arriving in bursts.

### 5.3.2 Mode-scoped validation

Validation rejects a frame only on the channels the active position mode consumes. In spherical mode those are joints one, two and four. A dead joint three, five, six or seven cannot reject a frame, which is the entire purpose: the arm keeps working with four of its seven channels gone.

A rejected frame holds the last good value rather than substituting a fabricated one. The cost is staleness rather than error, and it is charged honestly: a burst of dropouts freezes the command for the duration of the burst, which flattens measured displacement and adds apparent lag.

### 5.3.3 The degradation ladder

Figure 5.2 gives the measured cost of each reduction, against 20 430 recorded frames. Two orderings in it are not intuitive and both are mechanical rather than statistical.

| channels retained | mean tip error | capability                  |
|-------------------|----------------|-----------------------------|
| 4 pots            | 0 (reference)  | full position and reach     |
| 3 pots, no J3     | 41.6 mm        | roll lost, reach intact     |
| 2 pots, no J2 J3  | 29.2 mm        | shoulder lost               |
| 2 pots, no J3 J4  | 66.1 mm        | elbow lost, reach degraded  |
| 1 pot + 2nd IMU   | 63.8 mm        | elbow from IMU              |
| 0 pots            | n/a            | <b>POSITION UNAVAILABLE</b> |

Measured on 20 430 recorded frames from the left arm, against the shipped four-potentiometer reach. The error is charged to *reach magnitude* only, because in spherical form direction comes from the inertial measurement unit; scoring raw seven-joint forward-kinematic error instead would charge each reduction for a direction error the pipeline never incurs.

**Two orderings in this table are not intuitive and both are mechanical.** Dropping J3 costs 41.6 mm on the left arm and 4.5 mm on the right, because a roll carries no radial information on its own. And dropping J2 alone costs *less* than dropping J3 and J4 together, because the elbow dominates reach and the shoulder does not.

The last rung is the design decision that matters. With no well-conditioned channels the node reports position *unavailable* rather than extrapolating one, because a fabricated position on a manipulator bolted to a person is the worst available failure.

Figure 5.2: Capability against retained channels, with the cost of each rung measured on recorded hardware data.

The last rung is the design decision. With no well-conditioned channels the node reports position unavailable and publishes that state, rather than extrapolating a position from what remains. A fabricated position on a manipulator bolted to a person is the worst available outcome, and it is indistinguishable downstream from a real one unless the distinction is made explicit at the source.

### 5.3.4 Reduced-degree-of-freedom fallback

When both bend joints of an arm are unavailable, the forward-kinematic magnitude in Eq. (5.3) becomes constant and the arm's commanded set collapses from a shell of finite thickness to a surface. Elevation and azimuth are unaffected, because they are angular and come from elsewhere.

The fallback restores the missing dimension on a channel that is alive, coherent, and not otherwise consumed. Deflection of the wrist roll past a deadband commands a *rate* of change of radius rather than an absolute radius. Rate rather than position is necessary because that joint's usable range is small, so an absolute mapping would give coarse resolution over the whole radial extent and would require the operator to hold a wrist angle to hold a radius.

The channel choice is constrained. The gripper is on the force sensors and the clutch is on the buttons, so both are unavailable without conflict. The wrist roll conflicts with nothing, which means the operator can extend while

gripping and while indexing.

## 5.4 Control modes

Six operating modes are defined in a single registry, in the package that depends on nothing else, so every other package can read it without inverting the dependency graph. Table 5.1 summarises them; the four that are implemented and distinct are given their own data-path diagrams below.

Table 5.1: Operating modes. Autonomy runs slower than teleoperation by design: nobody is holding a master, so the only bound on a wrong motion is how long it takes to happen.

| Mode                | Input          | Robot decides               | $v_{\max}$ (rad/s) |
|---------------------|----------------|-----------------------------|--------------------|
| Direct, mannequin   | master arm     | nothing                     | 0.60               |
| Direct, VR          | Quest          | nothing                     | 0.60               |
| Orientation assist  | master, vision | wrist                       | 0.60               |
| Shared autonomy     | master, vision | wrist, target, approach     | 0.60               |
| Supervised autonomy | point or voice | and grasp, transport, place | 0.25               |
| Full autonomy       | voice          | and target                  | 0.15               |

Orientation assist is a stub and is marked as one. It would supply wrist orientation under an orientation constraint, and the inverse-kinematics plugin in use ignores the constraint field entirely: measured over 270 solver calls, success was identical to the digit with and without the constraint. It needs a constraint-aware plugin or explicit yaw sampling.

One safety stack serves all six. Every mode transition asserts the invariant, and dropping any one of the six shared mechanisms is refused in every mode rather than being available as a per-mode option.

### 5.4.1 Why the transport for VR is USB rather than the network

The headset connects over adb reverse on USB, so the client addresses `ws://127.0.0.1`. Browser-based immersive sessions require a secure context, which over a local network means a self-signed certificate carrying the address, a firewall exception, and an operator accepting a warning inside a headset they are wearing. A loopback address is a secure origin by definition, so all of that disappears. The link is also not the wireless network the arms are on.

### 5.4.2 Why the voice mode uses a grammar and not a language model

The command set is closed and small: grab, handover, place, stop. A quantised seven-billion-parameter model would occupy three to four of the four gigabytes of video memory available on this machine and add several hundred milliseconds, and it would place a non-deterministic component in the path that decides whether a robot beside a person's head begins moving.

The failure modes differ in kind, which is the substantive argument. A grammar that does not recognise an utterance returns unparsed, and the robot asks again. A language model asked for structured output can return a different, plausible verb, and nothing downstream can distinguish that from a correct parse. For five verbs that is a poor exchange.

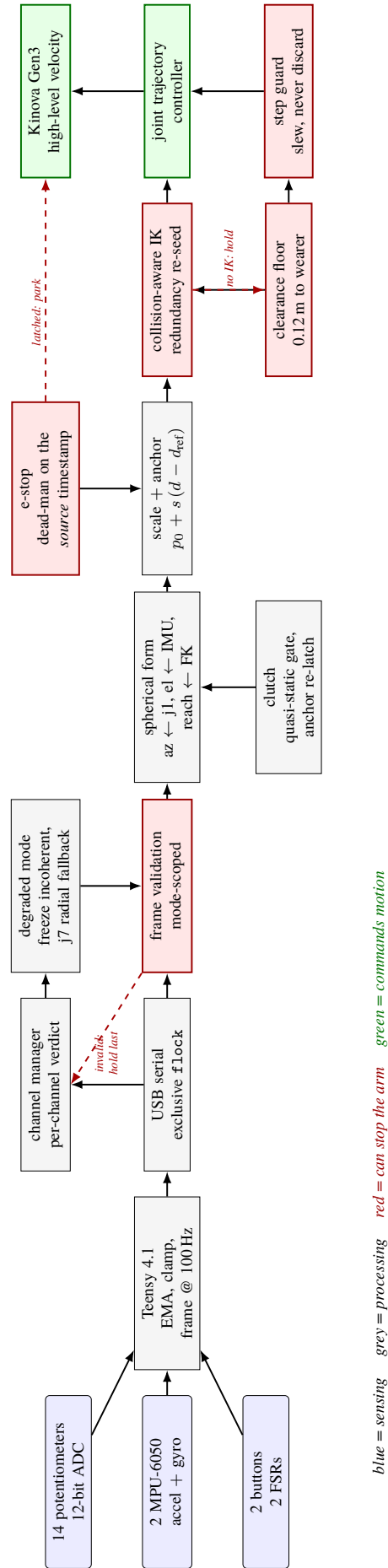
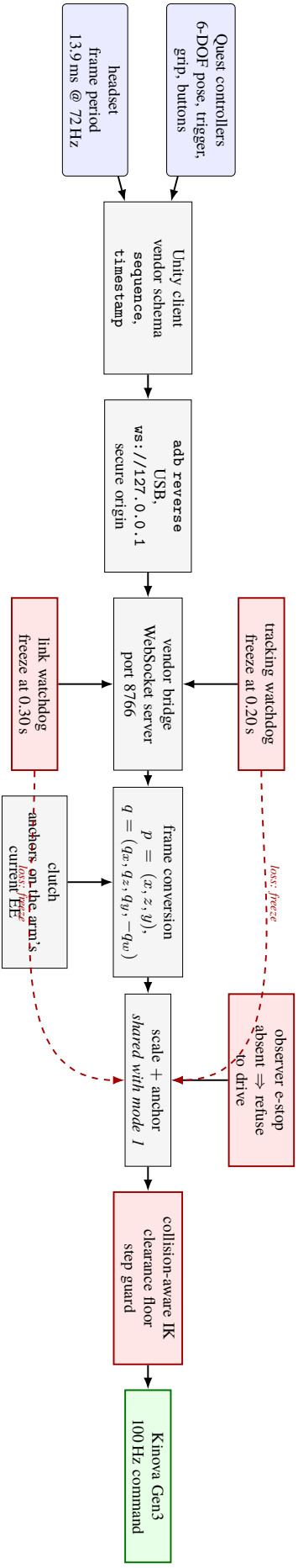
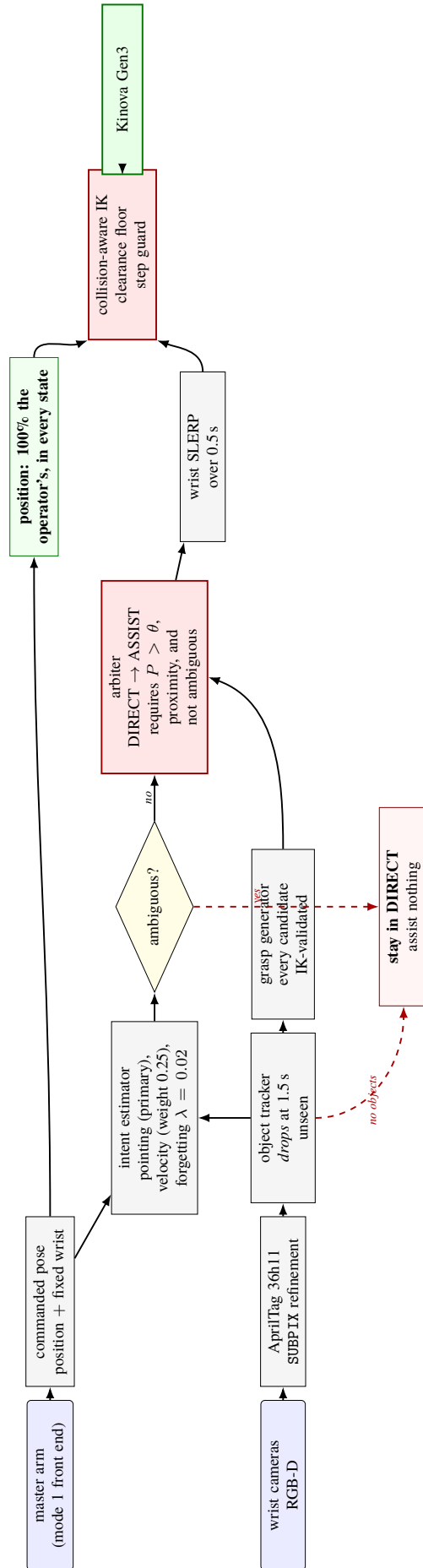


Figure 5.3: Direct teleoperation from the mannequin master. Red stages can stop the arm; dashed red arrows are failure exits.



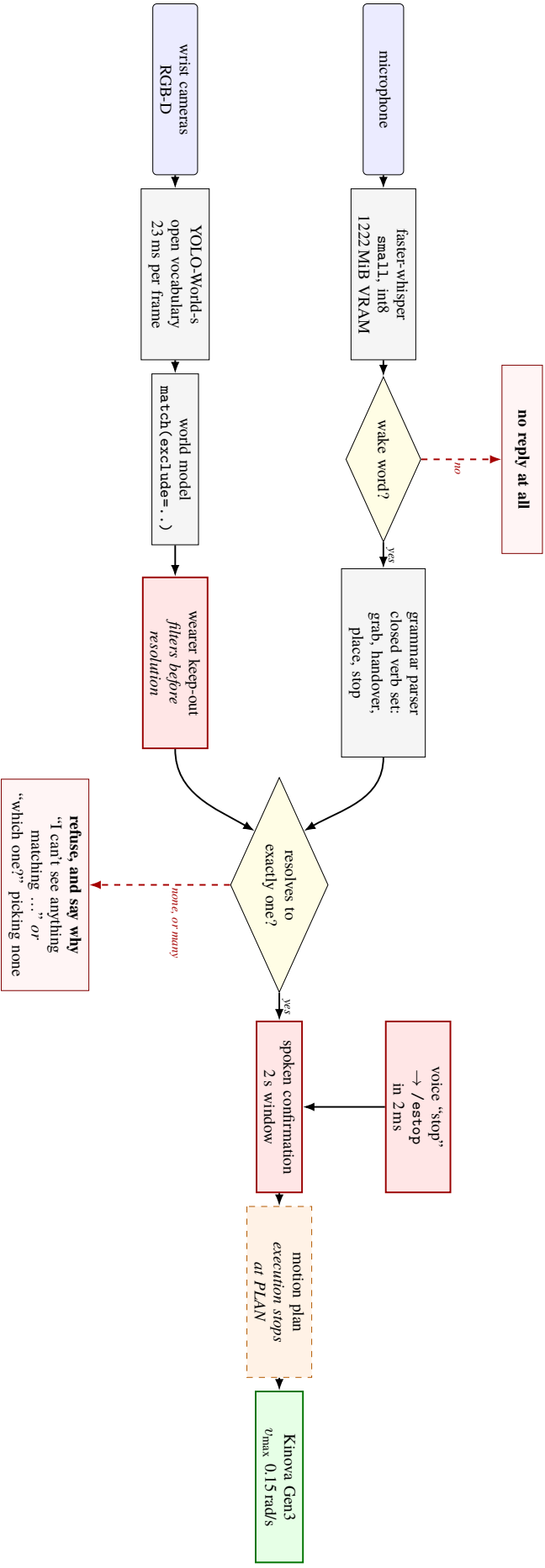
The headset supplies 6 DOF directly, so no spherical decomposition and no channel-health layer are needed. Everything from scale + anchor rightwards is the same code as Fig. 5.3.

Figure 5.4: Direct teleoperation from a Quest headset. The headset supplies six degrees of freedom directly, so neither the spherical decomposition nor the channel-health layer is required.



The green path carries position and never enters the autonomy stage, so no failure of intent estimation, object tracking or grasp generation can move the arm. Cancelling freezes the orientation blend where it is rather than snapping it back, which keeps the commanded pose continuous. The gripper is never closed by this mode. A tie between two adjacent objects holds  $P = 0.5$  exactly and the arbitrer stays in DIRECT, because assisting toward the wrong one of two candidates is worse than not assisting.

Figure 5.5: Shared autonomy. Position never leaves the operator, in any state.



*A no-wake utterance produces no reply at all: an earlier version answered "sorry, I didn't understand", which defeats the wake word by replying to conversations the robot was never addressed in. The keep-out filters candidates before disambiguation, so a target behind the wearer is never one of the options offered. Autonomy runs at a quarter of the teleoperation velocity cap because nobody is holding a master arm.*

Figure 5.6: Full autonomy from voice. Refusals are drawn as first-class outcomes because this mode declines more utterances than it accepts.

### 5.4.3 Why not an end-to-end vision-language-action policy

Three reasons, recorded so the decision is not silently reversed. Such models are trained on embodiment collections that contain no backpack-mounted dual Gen3, and zero-shot transfer to a novel embodiment is poor while fine-tuning needs demonstration data this project does not have. They emit actions directly, which bypasses every gate in this system, since collision-aware inverse kinematics, the clearance floor and the step guard all sit between a *pose* and the arm. And their failures are unattributable, where the modular pipeline fails at a named stage with its inputs logged.

## 5.5 Safety architecture

### 5.5.1 Collision avoidance in three stages

The wearer is in the collision model. Adding this was not a matter of setting a flag: the flag was already set, but the wearer’s model carried twenty-seven visual elements and one collision element, and no wearer link appeared in the semantic description. A planner cannot avoid what it cannot see.

Thirteen collision elements were added, paired with thirty-six exclusions for the mount-adjacent proximal links. The arm bases are bolted to the harness and are permanently adjacent, so without those exclusions every pose is trivially in collision. Everything distal of the first arm link stays enabled, which is what actually protects the wearer.

Avoidance is graduated. Collision-aware inverse kinematics rejects a configuration; on rejection the redundant degree of freedom is re-seeded and the solve retried, which recovers poses that would otherwise be dropped; and a geometric clearance floor measured from the transform tree provides a final margin. Driving the commanded pose directly at the wearer’s head and torso and holding it there stopped the arm at 0.072 to 0.079 m with zero clearance-floor blocks, so the earlier stages did the work and the floor remained a backstop. Every zone resumed on retreat.

### 5.5.2 Blocking must be observable

Four mechanisms in this system have historically stopped all motion silently. The common failure was never the blocking, which is usually correct. It was that blocking looked identical to working.

Every mechanism that can stop motion now registers with a monitor that enforces three properties: it announces itself by name on the first block and repeats; sustained blocking escalates in severity, because a block held for seconds is never normal; and a recovery string is mandatory at registration, so a blocker that cannot say how it clears is a latch and registration is refused.

### 5.5.3 Expiry is not permission

Every blocker in the system is written as a guard that blocks and returns, and clears on the fall-through path. That is correct while the guard is mutated elsewhere, and it latches the moment the guard also stops the callback running. This happened five times, the fifth inside the code written to catch the first four.

The structural fix is auto-expiry: a blocker not re-asserted for two seconds is treated as abandoned rather than held, since a genuinely held condition is re-asserted every cycle by design. Expiry is a third state, not a clear one.

Reading an expiry as clear would turn a crashed guard into a green light, which is worse than the latch it

Table 5.2: Blocker states. Expiry means the asserting loop stopped running, so the guarded condition is unknown rather than absent.

| State   | Meaning                                              | Treated as blocking |
|---------|------------------------------------------------------|---------------------|
| active  | the condition is held                                | yes                 |
| expired | the asserting loop stopped; condition <b>unknown</b> | <b>yes</b>          |
| clear   | live code says the condition is gone                 | no                  |

replaces. Two placement rules follow and both are load-bearing. The follower’s refusal to publish sits at the publish and not at callback entry, because the clears that resolve an expiry are upstream of it and refusing at entry would prevent the re-assertion that clears the unknown. And the unknown-state blocker is excluded from its own trigger, or it re-arms itself indefinitely.

#### 5.5.4 Emergency stop

Three independent triggers latch until an explicit reset: a service or topic call, both master buttons held simultaneously, and a dead-man on the master data going stale.

The dead-man keys on the *source* timestamp of the data, not on message arrival. This is the correction that made it work at all. The master node does not go silent when the master degrades: on a validation failure it substitutes the last good value and keeps publishing at full rate. It used to stamp that substituted frame with the publication time, so on the failure mode that actually occurs, a board emitting corrupted values rather than vanishing, the topic never went quiet and the timestamp never went stale. A dead-man keyed on arrival could not fire. Only a clean unplug produced silence, which is why every cable-pull test passed.

The first task of the set is a positioning task with three target widths at each amplitude, which characterises the interface against the speed-accuracy relation [6] and provides the single-arm against both-arms contrast that measures the attention bottleneck directly. [Figure 5.7](#) gives the design and [Fig. 5.8](#) the session structure.

## 5.6 Verification methodology

Four times in this project a result that looked like a system failure was a defect in the measuring tool. A noise floor of exactly zero on all fourteen channels was the recorder sampling faster than the sensor updates. An isotropy figure swinging by a factor of two between runs was a statistic chosen to be maximally sensitive to a rare tail failure. A detection rate of 0 to 4 % was a renderer producing images unlike anything the detector was trained on. A path-verification failure was the verifier carrying its test path in the one direction that leaves the feasible region.

The rule adopted is that a surprising failure is evidence about the instrument until the instrument has been cleared against a known answer. In practice this means every analysis script has a known-answer test, real recorded data is preferred to generated data, synthetic data is used only where the ground truth is constructed rather than rendered, and a result contradicting an earlier measurement is treated as an instrument check until the two are reconciled.

A corollary governs every coordinate in this thesis. The inverse-kinematics solver restarts randomly, so a pose at the edge of the feasible set is a coin flip and one call is one flip. Verification is therefore ten repeats over the whole path, densified to 20 mm, not two calls at the endpoints. Poses are graded solid, marginal or failed, and marginal is not usable.

**Chosen: 2 modes  $\times$  5 tasks, within subjects,  $n = 16$  dyads**

|                                     |                                               |                                        |                                            |                                          |
|-------------------------------------|-----------------------------------------------|----------------------------------------|--------------------------------------------|------------------------------------------|
| <b>T1 positioning</b><br>L, R, both | <b>T2 pick and place</b><br>one arm at a time | <b>T3 rigid carry</b><br>both, coupled | <b>T4 compliant carry</b><br>both, coupled | <b>T5 dual pursuit</b><br>both, disjoint |
| direct teleoperation                | direct teleoperation                          | direct teleoperation                   | direct teleoperation                       | direct teleoperation                     |
| shared autonomy                     | shared autonomy                               | shared autonomy                        | shared autonomy                            | shared autonomy                          |

Order is counterbalanced with a Williams square, which balances immediate sequence as well as position. Mode is within subjects because the between-subjects alternative needs 96 participants to reach the same sensitivity.

**Why not all four modes.** Four modes  $\times$  five tasks is 260 to 320 min of task time, a 5.5 to 6.5 h session, against a 48 min cap on how long a person can wear 17 kg. The three costed options were:

| option                       | conditions | session        | detectable effect                                              |
|------------------------------|------------|----------------|----------------------------------------------------------------|
| mode between subjects        | 5          | 127 min        | only $f = 0.40$ , needs $n = 96$                               |
| all modes, one task          | 8          | 169 min        | breaks the pack-on cap                                         |
| <b>two modes, five tasks</b> | <b>10</b>  | <b>167 min</b> | <b><math>d = 0.73</math> at 80% power, <math>n = 32</math></b> |

Dropping VR and full autonomy is a real loss of scope and is recorded as one. The comparison kept is the one where the literature disagrees with itself.

Figure 5.7: Two modes crossed with five tasks, with the options that were costed and rejected.

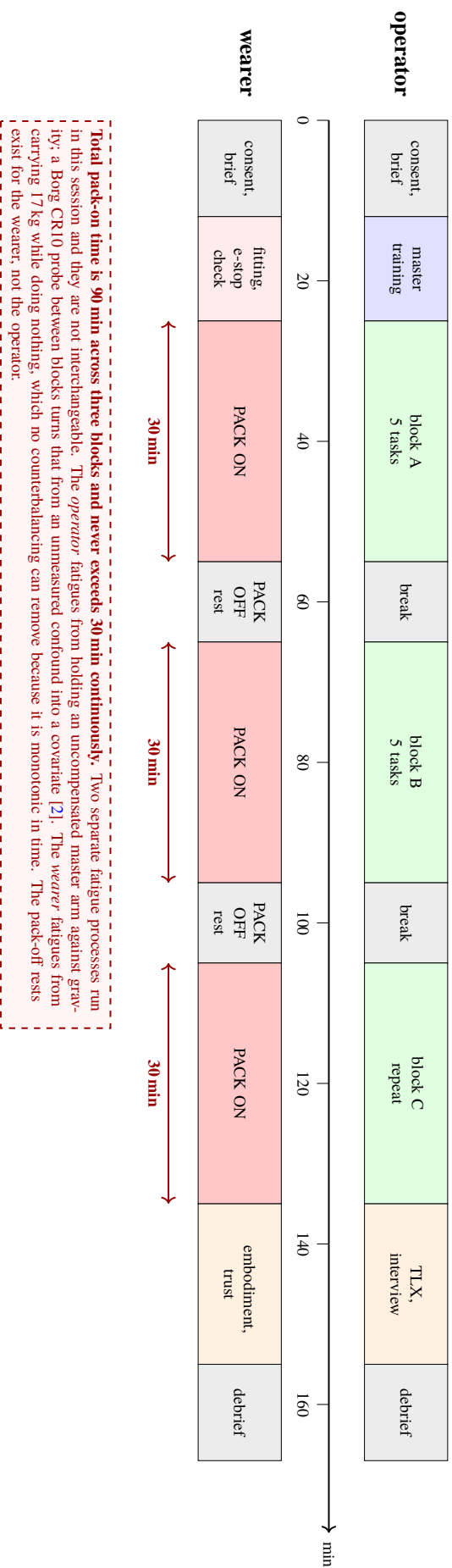


Figure 5.8: Session structure for two people. The wearer’s exposure is the binding constraint and is drawn as its own track.

## 5.7 Calibration

sample /master\_arm\_raw\_\*,  
never the port



**2. Zero reference**  
arm hanging; captures  
 $\hat{a}$  and the j1 datum



**3. Channel health**  
verdict per channel over  
*distinct* updates



**4. Range**  
sweep each joint to both  
mechanical stops



**5. Button map**  
check\_buttons: reports,  
never rewrites



**6. Workspace anchor**

**Step 0 is not optional.** Re-opening the serial port resets the board, and the IMU emits nonsense while settling. A gyro bias captured through a script that re-opens the port read 4.85 %/s of noise on the left sensor; sampled from the settled topic instead, the same gyro read 0.11 %/s. That single mistake was the difference between 1492 °/min and 44 °/min of estimated drift.

**Step 3 must precede step 4.** A range sweep on an incoherent channel measures the fault, not the joint: the readings span a wide interval without forming a trajectory, and range alone calls that healthy.

**PLACEHOLDER: IMAGE REQUIRED**

Annotated RViz screenshot at the home pose: both arms, the backpack mount, the wearer model with collision geometry shown, and the world frame triad. Must be labelled with the mount origin, the two end-effector frames, and the measured clearance from the nearest arm link to the torso.

Figure 5.10: Simulated platform at the home pose. Annotated screenshot required before submission.

# Chapter 6

## Results

Every result is labelled with the strength of evidence behind it. Three levels are used and the distinction is not cosmetic: *measured on recorded hardware data* means the numbers come from the physical master; *verified in simulation* means they come from the simulated platform with mock hardware; *implemented* means the code exists and runs but the quantity was not measured. Nothing in this chapter was validated on a physical Kinova arm.

### 6.1 Channel health

*Measured on recorded hardware data, 2026-08-06.*

Six of fourteen channels met the alive criterion.

Table 6.1: Per-channel verdicts, computed over distinct sensor updates. Baseline stored at recordings/baselines/channels\_20260806.json.

| Left arm |                              | Right arm |                          |
|----------|------------------------------|-----------|--------------------------|
| Channel  | Verdict                      | Channel   | Verdict                  |
| j1       | intermittent, 2.2 % dropouts | j1        | alive, 195°              |
| j2       | <b>incoherent</b> , 5 %      | j2        | alive, 261°              |
| j3       | <b>incoherent</b> , 8 %      | j3        | <b>incoherent</b> , 14 % |
| j4       | <b>incoherent</b> , 6 %      | j4        | alive, 291°              |
| j5       | <b>incoherent</b> , 12 %     | j5        | dead                     |
| j6       | <b>incoherent</b> , 8 %      | j6        | alive, 250°              |
| j7       | alive, 26°                   | j7        | dead                     |

Two entries reverse conclusions that had stood for weeks, and both reversals come from computing statistics over distinct updates rather than over transmitted rows.

Left j7 is the cleanest channel on that arm, with no dropouts and no implausible jumps. It had been recorded as railed and dead. The evidence for that verdict was a spread of exactly zero over forty consecutive samples, which is what an oversampled sensor produces regardless of its health. Its range is only 26°, so it is mechanically restricted, but it tracks.

Five of the seven left channels are incoherent, which is worse than had been recorded. These channels were previously assessed by range, and range calls them healthy: they return values spanning a wide interval. What they do not return is a trajectory.

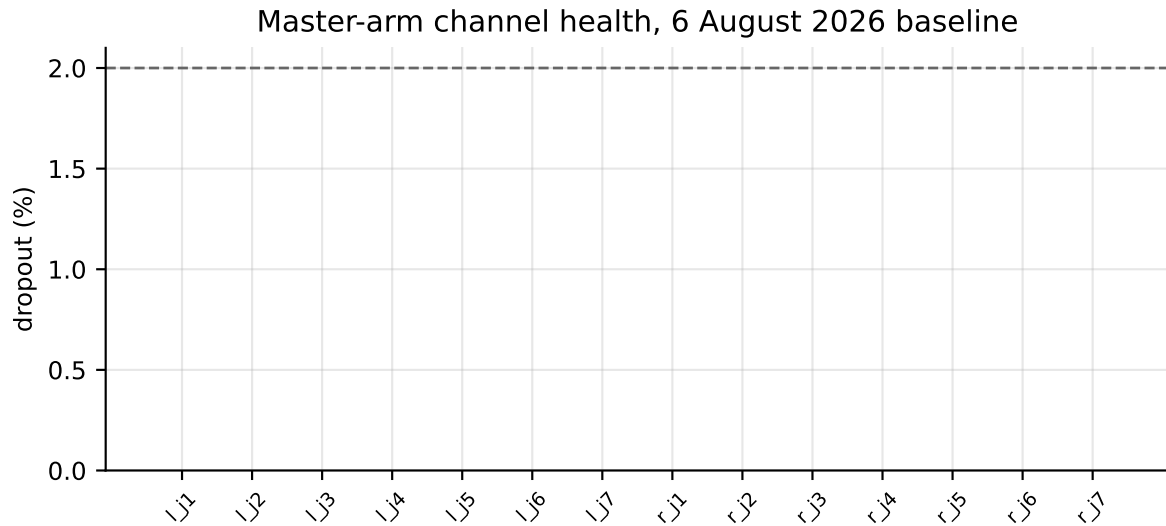


Figure 6.1: Per-channel dropout rate and jump rate, with the verdict boundaries marked.

### 6.1.1 Progressive degradation between two dates

*Measured on recorded hardware data, two captures.*

The two dated captures show the failure population growing rather than static, which is the empirical support for the premise in [Chapter 1](#) that channel loss is an operating condition. [Figure 6.2](#) gives the comparison.

## 6.2 What the degradation architecture costs

*Measured on recorded hardware data, 20 430 frames.*

[Figure 5.2](#) gave the ladder. The figures are commanded tip error against the four-potentiometer reference, charged to reach magnitude only, because direction comes from the inertial sensor and scoring raw seven-joint forward-kinematic error would charge each reduction for a direction error the pipeline does not incur.

Dropping the upper-arm roll costs 41.6 mm on the left arm and 4.5 mm on the right. Dropping the shoulder bend alone costs less than dropping the upper-arm roll and elbow together, at 29.2 mm against 66.1 mm. A second inertial sensor supplying the elbow reaches 63.8 mm on a single potentiometer, comparable to retaining two.

### 6.2.1 Which channels a repair should target

*Measured on recorded hardware data.*

Regressing the true four-potentiometer reach on every channel that still passes the coherence test, plus the inertial sensor, identifies the repair precisely.

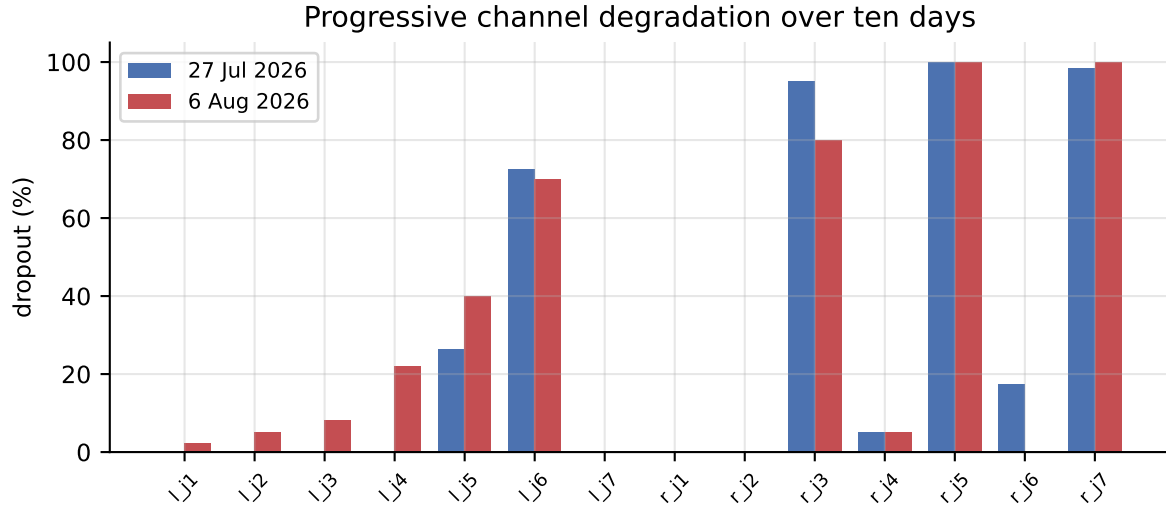


Figure 6.2: Channel verdicts across two dated captures. The population of failed channels grows.

Table 6.2: Reach observability from the surviving channels.

| Arm   | Live predictors          | $R^2$ | Residual RMS |
|-------|--------------------------|-------|--------------|
| Left  | j1, j7, inertial         | 0.133 | 51 mm        |
| Right | j1, j2, j4, j6, inertial | 0.986 | 9.1 mm       |

The left arm retains no reach observable at all: the residual is 93 % of the true spread. This is expected mechanically rather than surprising. Reach is set by shoulder flexion and the elbow; j1 and j7 are rolls about the arm's own axis and carry no radial information; and one wrist inertial sensor cannot observe the elbow. The right arm needs no help.

That names the repair as exactly two channels, left j2 and left j4, and predicts that repairing any others will not restore reach.

### 6.2.2 What the incoherent channels inject

*Measured on recorded hardware data, replayed.*

Replaying the recorded incoherent traces with every coherent channel and the accelerometer pinned to its median simulates a perfectly steady hand on perfect wiring, so every millimetre of output comes from the broken channels alone.

Table 6.3: Commanded end-effector motion contributed by the incoherent channels alone, with the operator's input held constant.

| Arm   | Degraded mode off            | Degraded mode on |
|-------|------------------------------|------------------|
| Left  | 54.527 mm standard deviation | 0.000 mm         |
| Right | 0.014 mm                     | 0.000 mm         |

The on column is zero by construction, since freezing a channel removes its contribution. The off column is the measurement: 54.5 mm of commanded motion that the operator did not ask for, on an arm whose whole master reach is 272 mm.

### 6.2.3 The pipeline manufactures no motion of its own

*Verified in simulation, against a synthetic serial device.*

Read alone, the previous result does not separate the channels from the software. A synthetic serial device streaming frames in the exact wire format at a known constant input closes that gap. With the master idle for 60 s and one stack running, commanded end-effector standard deviation was 0.0000 mm on both arms, with a worst simulated joint excursion of  $0.0051^\circ$ .

The two results are complementary. The synthetic device shows the pipeline is clean; the replay shows what the physical channels inject through it.

## 6.3 Command mapping

### 6.3.1 Axis correctness

*Measured on recorded hardware data, and verified in simulation.*

Regressing commanded pose against the master's physical spherical position over all sweep segments, at 6443 samples with  $R^2 = 0.999$ , returned a gain matrix of  $\text{diag}(+1, -1, -1)$  against the intended identity. The pipeline was faithfully implementing an axis map that was wrong: raising the master drove the arm down and moving it forward drove the arm back.

The correction is not a free choice of sign. In spherical form the elevation convention already carries the vertical sign, so any axis map with a negative vertical entry double-negates it, and no such map can produce correct behaviour. After correction, raising the master by  $30^\circ$  moves the arm up.

### 6.3.2 The lateral axis remains wrong, and the axis map cannot fix it

*Measured on recorded hardware data.*

Up and down and fore and aft track correctly on both arms. Lateral does not.

Fitting the best rotation from measured sweep directions to intended directions leaves a residual of  $54.8^\circ$  RMS on the left arm and  $54.7^\circ$  on the right. A rotation-invariant test explains why no rotation will help: intended left-right and forward-back are  $90^\circ$  apart, and measured they are  $137.7^\circ$  apart. A rotation preserves angles, so no rotation and no axis permutation maps one triad onto the other. The measured directions are not an orthogonal triad.

The mechanism is a lever arm, and identifying it correctly took two attempts. Joint one is the first joint and its axis is fixed in the mount, so it does not tilt as the arm raises. What changes is the offset of the tip from that axis: rolling joint one with a straight arm moves the tip by exactly zero, and produces lateral motion only once a downstream bend offsets the tip. Azimuth per degree of joint one therefore scales with how bent the arm is, so azimuth and elevation are not independent, which is precisely what the spherical model assumes.

Replacing azimuth with the forward-kinematic horizontal angle was measured and made things worse, from  $12.6^\circ$  to  $33.3^\circ$  mean reversal error on the left arm. Gyroscope-derived azimuth halves the residual on the left arm, from  $25.6^\circ$  to  $13.5^\circ$ , and is implemented but not enabled: the right sensor drifts at  $44^\circ/\text{min}$  while stationary, and three directions from one segment each does not establish a triad.

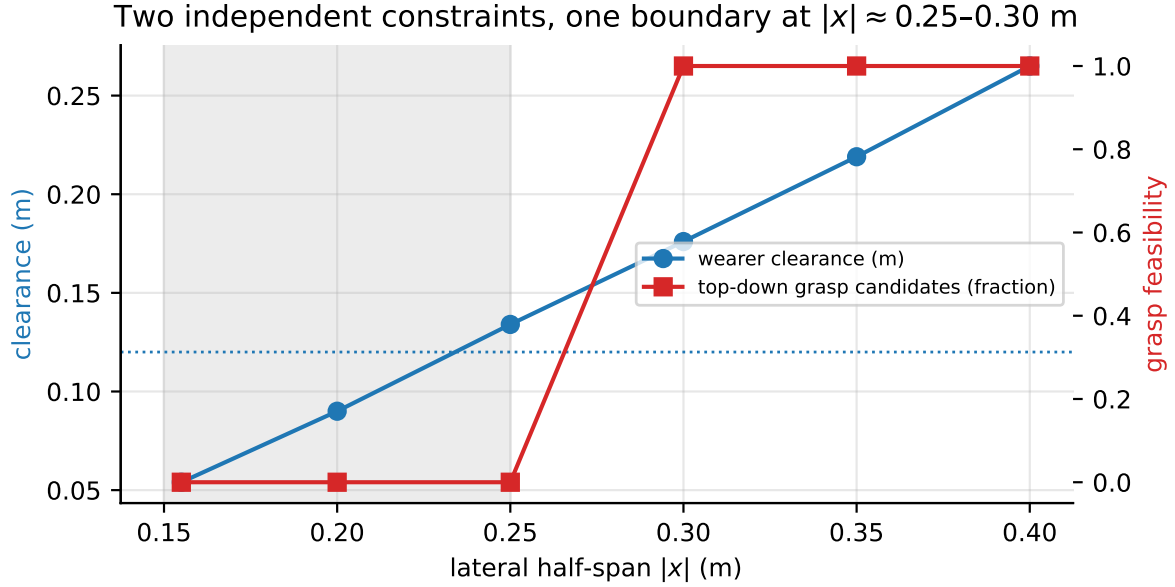


Figure 6.3: Measured against intended sweep directions. The pairwise angles cannot be reconciled by any rotation.

### 6.3.3 Indexing

*Verified in simulation, against the synthetic serial device.*

Ten index cycles gave a re-engagement jump of 0.29 mm mean and 0.35 mm maximum, drift while disengaged of 0.000 mm, and total travel of 338.8 mm over six cycles with the second half still advancing. The reachable set is bounded by the robot, not by the master's range.

The first run of this measurement showed four consecutive cycles of zero travel, which was a real defect rather than a measurement artefact: the radial fallback had integrated to its clamp, so further wrist deflection did nothing and said nothing. The clamp now warns by name. This is the silent-blocking class exactly, and the fix is not to remove the clamp but to make reaching it audible.

## 6.4 Safety

### 6.4.1 Fault injection

*Verified in simulation, fourteen injected faults.*

All fourteen were handled: channel dropout, board disconnect, board reconnection, emergency stop during motion, wrong-object latching, ambiguous target flicker, stale object, detection failure, logging failure, identifying data in a manifest, non-monotonic clock, session loss, zero camera detections, and network dropout.

Two of these took several attempts and the reasons are transferable. The dead-man did not trip on ten seconds of total master silence, which was diagnosed at the time as a broken dead-man and was not. The master republishes stale data at full rate, so the topic never went silent; the mechanism was keyed on the wrong quantity. And the emergency stop's named block did not appear in the aggregated view precisely when the dead-man fired, because the follower set that blocker only inside its pose callback, which stops being called at the moment the master goes quiet.

## 6.4.2 Dead-man timing

*Verified in simulation.*

Against a deadline of 1.00 s:

Table 6.4: Dead-man behaviour by failure mode.

| Case                                                              | Result                        |
|-------------------------------------------------------------------|-------------------------------|
| Healthy master, fresh timestamps, 243 frames                      | no trip                       |
| <b>Degraded: publishing at full rate, frozen source timestamp</b> | <b>trip at 0.94 to 0.99 s</b> |
| Clean unplug, topic silent                                        | trip at 0.97 s                |

The middle row is the one that could not have passed before the correction, and it is the failure mode that actually occurs.

## 6.4.3 An emergency stop that was four seconds late

*Verified in simulation.*

Detection was always on time. The halt was not. The trip handler called the real-driver shutdown first, which waited on two services that do not exist on a simulated stack, so both timed out in full inside the node's own timer. Measured: decision at 0.97 s, trip logged at 4.98 s. The arms were parked four seconds after the decision to park them, and the log line that would have revealed this was itself behind the block.

The fix parks first, using only local publishes, and makes the driver path non-blocking.

## 6.5 Grasping

*Verified in simulation.*

The grasp sequence executes a pre-grasp standoff with the wrist pre-aligned, a straight approach along the approach vector, closure to the object's width, retreat along the same vector, transit, and release at the placement. Every candidate is validated at the standoff, at the grasp, and at every point of the approach between them before it is offered.

Two defects were found in the recorded gripper trace rather than by inspection, and neither was visible in the source.

Two components were writing the gripper on the same cycle. The recorded knuckle angle sawed through the lift phase, from 0.42 to 0.18 to 0.24, and reached 0.05, fully open, at the start of transit: the object was grasped, dropped, and re-grasped in mid-air when the second component's schedule crossed its own threshold. After giving the gripper a single owner, the lift phase holds 0.424 with minimum equal to maximum over eighteen samples, and there is exactly one release, at the placement.

The object marker attached on a band rather than on the object's width, so a 40 mm block requiring a knuckle angle of 0.424 attached at 0.157, with the fingers still visibly open. Attachment now requires 0.90 of the width-specific target and occurs at 0.399.

### 6.5.1 Top-down grasping has a lateral boundary

*Verified in simulation.*

Top-down grasp candidates fail entirely at  $|x| = 0.25$  m, at zero of nine, and succeed entirely at  $|x| = 0.30$  m, at nine of nine. The failure persists with collision checking disabled, so it is wrist orientation infeasibility and not obstruction. Every pick point in the task set is placed beyond that boundary as a result.

## 6.6 The wrist cannot be aligned under teleoperation

*Verified in simulation, and a limitation rather than a result.*

The default orientation mode pins the commanded wrist to the anchor captured at the home pose. For the task-set block at  $(0.32, 0.35, 1.15)$ , the top-down grasp requires a rotation of  $169.7^\circ$  from that anchor, which is very nearly a reversal.

There is no channel to command it. The master's wrist is unmeasurable, with left  $j7$  restricted to  $26^\circ$  and  $j6$  incoherent, and right  $j3$ ,  $j5$  and  $j7$  dead or incoherent, which is why the mode defaults to pinned. Orientation assist, which exists to supply it, is a stub because the solver plugin ignores orientation constraints.

The recorded grasp sequences work because the recorder calls the inverse-kinematics service directly with the grasp orientation, bypassing the teleoperation orientation lock. They demonstrate that the *robot* can execute an aligned grasp. They do not demonstrate that an *operator* can command one, and the three recorded conditions differ only in trajectory smoothing, so they do not distinguish this either.

This is the one place where autonomy earns a categorical rather than a quantitative advantage over direct teleoperation on this platform, and it should be claimed exactly that narrowly.

## 6.7 Task-set verification

*Verified in simulation, ten repeats over densified paths.*

Every coordinate of the five-task set was verified at ten repeats over paths densified to 20 mm, with both arms confirmed at 0.0000 rad from the home pose before any sample was taken. Zero failures.

The home assertion earned its place. An earlier run of the same script with the arms parked 0.87 rad and 0.59 rad from home reported zero failures as well, and was measuring a different question under the same name. With the arms genuinely at home, ten of twenty-five coupled waypoints failed, which is what forced the coupled-object respecification in [Chapter 7](#).

Table 6.5: Five-task verification. Both arms verified at home; ten repeats per pose; short-circuit on first failure.

| Task            | Coverage                            | Result       |
|-----------------|-------------------------------------|--------------|
| Positioning     | 3 targets per arm                   | 3/3, 3/3     |
| Pick and place  | 3 picks per arm, approach and place | 3/3, 3/3     |
| Rigid carry     | 3 paths, 25 densified waypoints     | all verified |
| Compliant carry | as rigid carry                      | all verified |
| Dual pursuit    | 26-direction shell per arm          | 26/26, 26/26 |

The result reproduces. Re-running the verification returned a result identical to the previous run, field for field. That is the first reachability figure in this project to reproduce across runs rather than be re-asserted, and it reproduces because the respecification moved every coupled waypoint off the feasibility boundary.

## 6.8 Performance figures

*Verified in simulation. Every latency below is control-path latency and is a lower bound on end-to-end latency, since it excludes the operator, the display and the physical arm.*

Table 6.6: Measured rates and latencies. All against mock hardware.

| Quantity                | Value                       | Note                          |
|-------------------------|-----------------------------|-------------------------------|
| Master frame rate       | 100 Hz                      | firmware loop                 |
| Distinct sensor updates | 14.7 to 17 Hz               | see <a href="#">Chapter 3</a> |
| VR packet rate          | 68.5 Hz                     | 822 packets over 12.0 s       |
| VR round trip           | 8.66 ms median, 16.5 ms p95 | client-timed, one clock       |
| Voice stop to /estop    | 2 ms                        |                               |
| Console frame time      | 0.01 ms median, 4.85 ms max | against a 33 ms budget        |
| Kortex high-level loop  | 18.4 to 18.7 Hz             | two round trips per cycle     |

The Kortex figure is the one that constrains the architecture. The cyclic control path is designed for a 1 kHz loop on a dedicated link, and over the virtualised network layer on this machine each cyclic write is a round trip costing about 10 ms. At 100 Hz the write alone consumes the entire cycle budget, the controller manager overruns permanently, and no command reaches the arm while feedback continues to arrive perfectly. The symptom is an arm that appears dead while serving live telemetry. The system therefore uses the high-level velocity interface, which holds a commanded motion between sends and so does not require kilohertz updates.

### PLACEHOLDER: IMAGE REQUIRED

Screenshot of the operations console under load, showing the pipeline view with one stage marked as the root cause and downstream stages marked as consequences, the per-channel staleness panel, and the blocking summary. Must be captured with a fault present so the first-out annunciation is visible.

Figure 6.4: The console during a fault. Screenshot required before submission.

## 6.9 What is not measured

The following are implemented and unmeasured, and none should be read as working.

- **Anything derived from a force reading.** The mock hardware returns identically zero for every force, so payload compensation, interaction force and grip force at the slave are *structurally undefined* rather than merely unmeasured. Comparing tracking with and without a payload configured returned a difference of exactly 0 m, which is a property of the mock and not evidence.
- **Object detection rate at working distance.** The models fit in the available video memory at 2012 MiB of 4096 MiB, but the synthetic renderer used to measure detection produces images outside the detector's training distribution. The same detector scores 0.89 to 0.91 on a real photograph and zero on the synthetic task objects, so the measurement characterises the renderer. The detection gate is neither passed nor failed.

- **Grasp success in the autonomous modes.** Execution stops at the planning stage.
- **Everything on real arms.** Two Kortex sessions have been opened simultaneously and homing has completed against mock hardware only.

# Chapter 7

## Workspace characterisation and its consequences

This chapter is separated from [Chapter 6](#) because it is not a single measurement but a chain of them, and because its consequences determined the shape of the task set. It reports a negative finding together with a quantified and buildable mechanical fix; the finding alone would be a complaint rather than a contribution.

### 7.1 Continuous reachability from the home pose

Reachability was measured as a swept extent rather than as pointwise feasibility, because teleoperation requires the arm to travel to a pose from where it already is, without a branch change. Twenty-six directions were swept in 1 cm steps from the home configuration, each step seeded from the previous solution exactly as the follower does.

Repeating the sweep ten times gave identical quartiles: the inter-quartile range was 0.000 m in every direction for both arms. The measurement repeats. What did *not* repeat in earlier runs was the summary statistic: isotropy taken as the minimum over directions of a single sweep is dominated by rare early terminations and varied by a factor of two between runs. On medians it is stable, at 0.16 for the left arm and 0.10 for the right ([Fig. 7.1](#)).

The early terminations were traced to the measuring tool rather than to the robot. A sweep is a chain of up to ninety solver calls and any single failure ends that direction; the follower does not suffer this because it re-seeds the redundant joint and retries. Giving the sweep the same retry reduced early terminations from nine to zero on the left arm and three to zero on the right, while the medians barely moved. That the medians were already right is what establishes the outliers as chain compounding rather than geometry.

### 7.2 The two arms share no reachable workspace

Over a grid of sixty-three cells spanning the frontal working volume, with the arms verified at the home pose, **no cell was reachable by both arms**. The result survives the two obvious objections, as [Table 7.1](#) shows: freeing the wrist yaw raises each arm's individual reach from sixteen and eighteen cells to twenty-one and still yields no shared cell, and repeating the test with more redundancy retries changes nothing.

A fine sweep in 0.05 m steps across the plane on which the arms would have to meet locates the boundary precisely: the right arm reaches  $x \leq -0.10$  m, the left reaches  $x \geq 0.15$  m, and there is a 0.25 m dead band between them in which neither arm can place its end effector ([Fig. 7.2](#)).

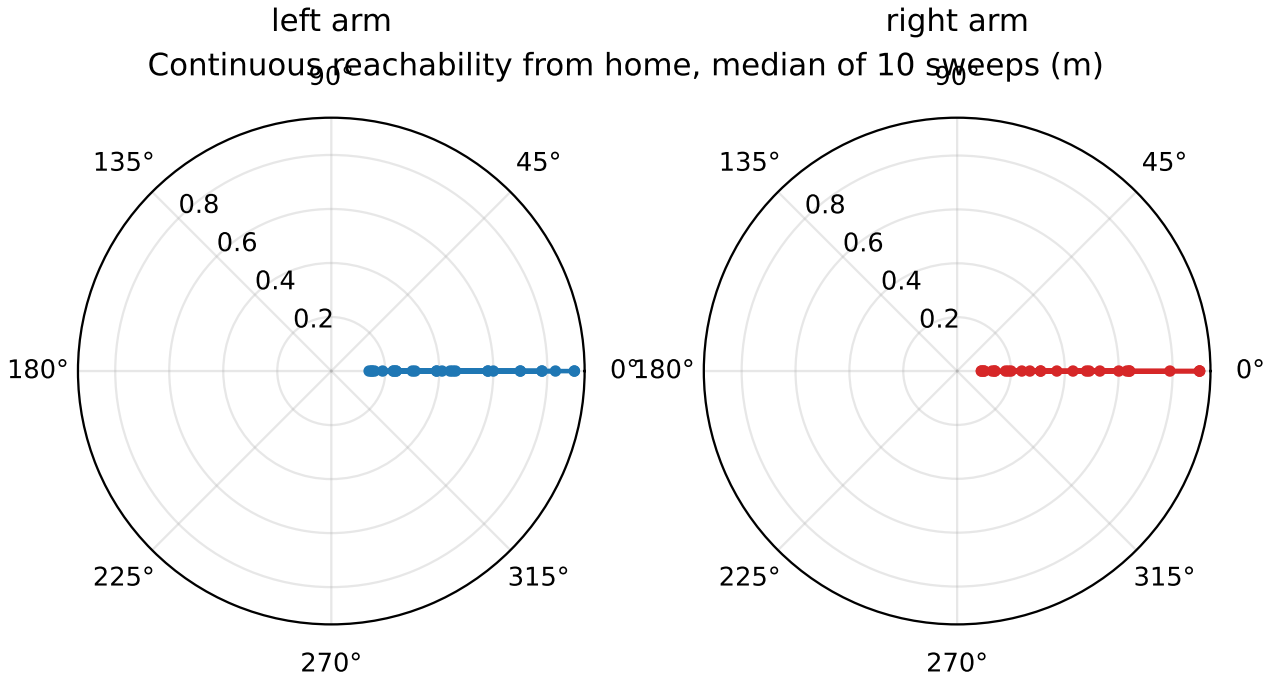


Figure 7.1: Continuous reachability from the home pose, median of ten sweeps, projected onto the horizontal plane. The envelope is markedly anisotropic: the median reach is 0.34 to 0.40 m but the worst direction is 0.14 m on the left arm and 0.09 m on the right.

Table 7.1: Both-arm overlap over the 63-cell frontal grid, arms verified at home. The null result survives both wrist freedom and additional inverse-kinematics retries.

| Test condition                          | Both        | Left | Right |
|-----------------------------------------|-------------|------|-------|
| Fixed orientation, 3 retries            | <b>0/63</b> | 16   | 18    |
| Fixed orientation, 6 retries, 3 repeats | <b>0/63</b> | 17   | 18    |
| Yaw free, 6 retries                     | <b>0/63</b> | 21   | 21    |

The two reachable sets do not meet (measured, N=3, yaw-free)

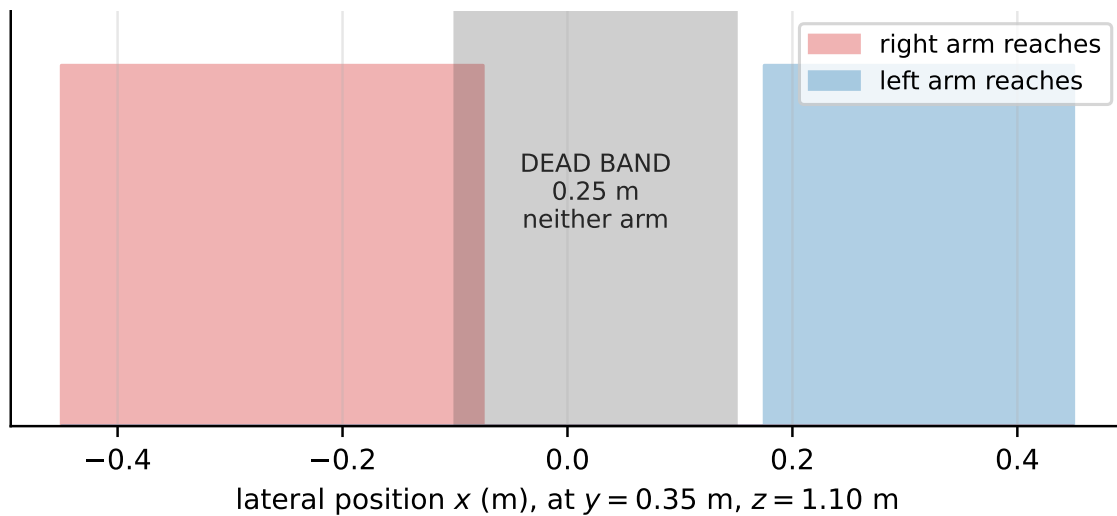


Figure 7.2: The two reachable sets do not meet. Measured at  $y = 0.35$  m,  $z = 1.10$  m with yaw free and three repeats per point.

### 7.2.1 What this does and does not establish

It establishes that, within  $x \in [-0.6, 0.6]$ ,  $y \in [0, 0.4]$ ,  $z \in [0.90, 1.30]$  m, with the as-built mount and the legacy home joint angles, the two reachable sets do not intersect.

It does *not* establish behaviour outside that volume, nor does it generalise to other mounts. The correct claim is that the arms share no workspace *as mounted and as parked*, and the remedy is mechanical.

## 7.3 The fix is buildable

A parametric sweep over candidate mount placements shows the null result is not intrinsic. Moving the mounts inboard, forward and down, with additional forward tilt, raises the number of shared cells from three to **twenty-two** on the purely kinematic measure, within a buildable 0.20 m mount separation (Fig. 7.3).

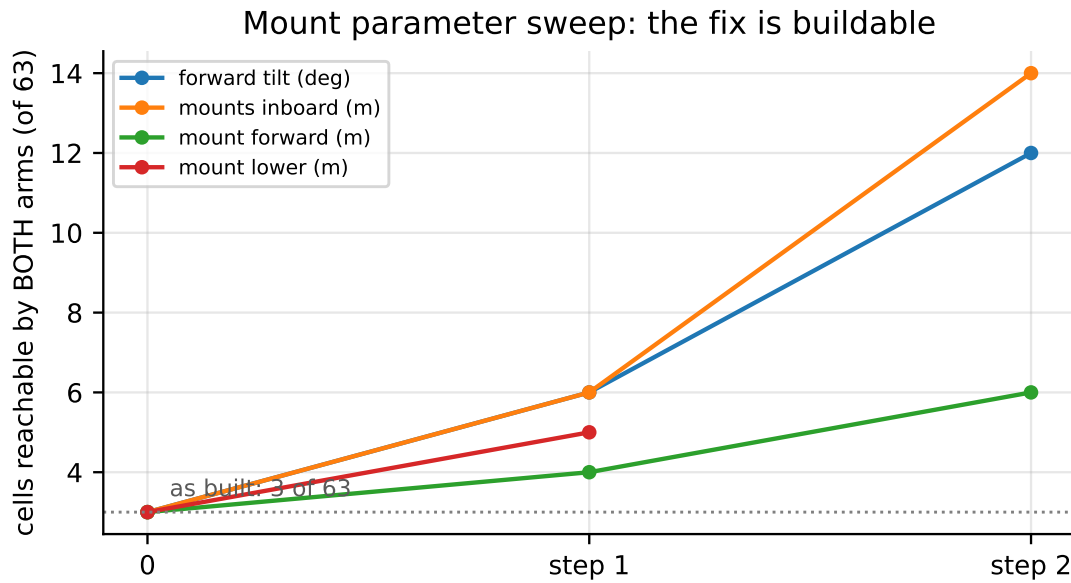


Figure 7.3: Single-parameter sweeps from the as-built mount. Forward tilt and inboard translation both help substantially; splaying the mounts outward *hurts*. The sweep is purely kinematic, moving the arms’ bases without moving the wearer, so these are upper bounds and any candidate requires a clearance check before adoption.

The sweep was recorded and deliberately not applied, because changing the mount invalidates the home pose, the workspace anchor and every clearance figure reported in Chapter 6.

## 7.4 One boundary, two independent constraints

Two further measurements, taken for different reasons, land on the same lateral boundary and together make the case that the inboard band is unusable.

The first is wearer clearance. Every scenario in the chest-height band passes inverse kinematics with collision checking enabled, and the planner reports the poses valid, yet the arms pass within 48 to 67 mm of the wearer’s torso, inside the 120 mm margin the real-robot configuration enforces. The reconciliation is that the two tests ask different questions: the planner’s check is binary contact, whereas the floor is a margin, and a pose can satisfy one while being refused by the other. Sweeping the lateral half-span shows clearance rising monotonically and crossing the floor at 0.25 m.

The second is grasp feasibility. A top-down grasp requires the wrist to adopt an orientation with its approach axis vertical. Sweeping the same lateral coordinate gives a sharp cliff: zero of nine grasp candidates are solvable at  $|x| = 0.25$  m and nine of nine at  $|x| = 0.30$  m. The failure persists with collision checking disabled, so it is kinematic infeasibility of the wrist orientation rather than obstruction.

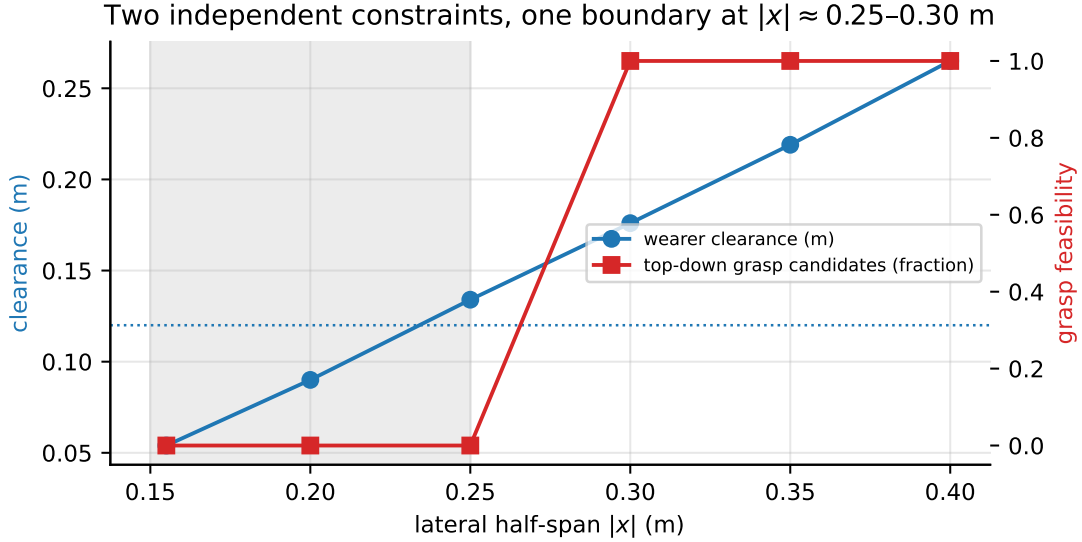


Figure 7.4: Wearer clearance and top-down grasp feasibility against lateral half-span. The two constraints were measured independently and for different purposes, and both place the usable boundary at  $|x| \approx 0.25$  to  $0.30$  m.

## 7.5 Consequences for the task set

Three consequences follow directly, and each is visible in the task definitions of [Chapter 6](#).

**Inter-arm handover is impossible.** It requires both arms at one point, and no such point exists. This removes an entire task family, and no control, planning or autonomy change can restore it.

**A bimanual task must either span the gap or exploit simultaneity.** A rigid object held at two points can bridge the dead band; two targets in the two disjoint sets can require simultaneous satisfaction. Both routes are used.

**The coupled objects had to be re-specified.** The original 310 mm tray placed each grip at  $|x| = 0.155$  m, inside the dead band, and ten of twenty-five waypoints failed once the arms were verified at home. Widening to 500 mm places each grip at  $|x| = 0.25$  m, which satisfies the clearance boundary and the reachability boundary simultaneously. The compliant carrier had to lengthen with it: retention of a ball of radius  $r$  requires sag of at least  $2r$ , so

$$\text{sag}(s) = \sqrt{\left(\frac{L}{2}\right)^2 - \left(\frac{s}{2}\right)^2}, \quad s_{\max} = 2\sqrt{\left(\frac{L}{2}\right)^2 - (2r)^2}, \quad (7.1)$$

where  $s$  is the gripper separation,  $L$  the carrier length and  $r$  the ball radius. For  $L = 540$  mm and  $r = 20$  mm this gives  $s_{\max} = 534$  mm, leaving a 34 mm margin at the nominal 500 mm separation ([Fig. 7.5](#)). The length is chosen so that the failure threshold sits *inside* the reachable band: a longer carrier could never fail and would measure nothing.

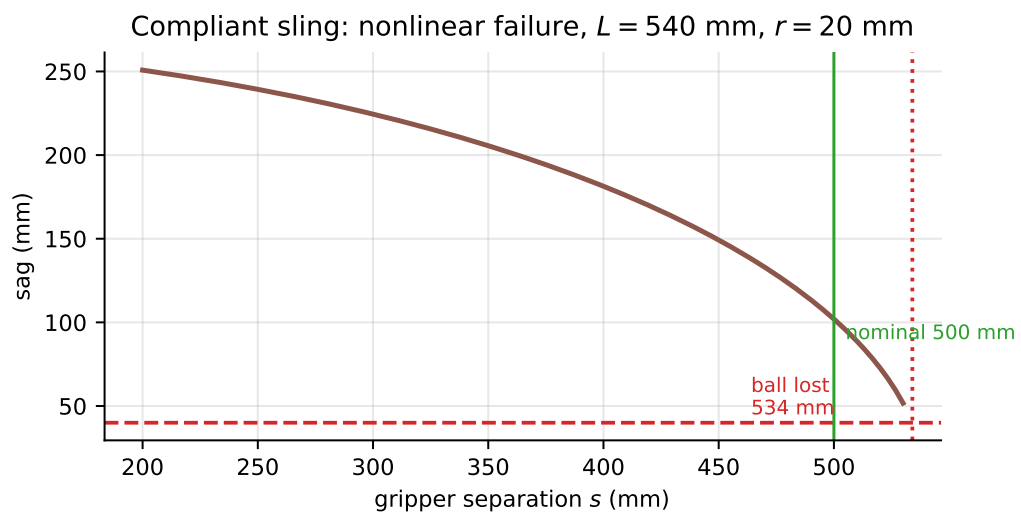


Figure 7.5: Sag against gripper separation for the re-specified compliant carrier. The nonlinearity is the point: sag falls slowly, then the ball is lost abruptly at 534 mm.

# Chapter 8

## Discussion

### 8.1 What the degradation architecture bought, and what it did not

The premise in [Chapter 1](#) was that a body-worn master will lose channels, so losing them must be survivable. The two dated captures support the premise empirically: the failed population grew between them ([Section 6.1](#)). The architecture's value can be stated precisely from the measurements.

It kept the system running. With eight of fourteen channels failing the health criterion, the right arm retains full position sensing at  $R^2 = 0.986$  and the left arm retains direction. A joint-space mapping would have been inoperable on either arm from the first failed channel, because it has no reconstruction path.

It kept the failures visible. The incoherent verdict exists because a failing potentiometer does not go quiet, and range alone calls a wide-spanning nonsense signal healthy. Without that verdict the system would have kept following 54.5 mm of commanded motion the operator never asked for, on a master whose whole reach is 272 mm.

What it did not buy is capability. Freezing the left arm's bend joints removes its radial dimension entirely, collapsing its commanded set from a shell to a surface, and the wrist-roll fallback restores an axis rather than restoring the measurement. The architecture converts a fault into a known, bounded, announced reduction. It does not convert it into working hardware, and [Section 6.1](#) names the two channels whose repair would.

### 8.2 The reduced-degree-of-freedom premise, assessed

The second premise was that commanding fourteen joints through a body-worn master is a usability problem, and that the answer is to take each component from the sensor that observes it best. The evidence is mixed, and it divides cleanly by component.

Elevation from gravity works. It is mount-independent, drift-free, integrates nothing, and survived every reformulation. Reach from the forward-kinematic magnitude works, because a magnitude depends on the bend joints and tolerates a dead roll passed through as zero. Both vertical and fore-and-aft motion track correctly on both arms.

Azimuth from a single joint does not work, and the reason is instructive rather than incidental. The spherical model assumes azimuth and elevation are independent. They are not, because joint one is a roll about the arm's own axis, so its effect on the tip scales with how bent the arm is. The  $137.7^\circ$  measured between two directions intended to be  $90^\circ$  apart is not a calibration error that a better axis map or a better zero could remove; it is the model being wrong about the geometry.

The honest reading is that the decomposition succeeded for the two components where a single sensor observes

the quantity directly, and failed for the one where it does not. That is a usable design rule for a successor: decompose along the axes your sensors actually observe, not along the axes that make the mathematics tidy.

### 8.3 The workspace result and what it costs the study

[Chapter 7](#) reports that the two arms share no reachable workspace under the as-built mount. The consequence for the experimental programme is severe and worth stating plainly rather than absorbing into a limitations list.

Inter-arm handover is impossible. Not difficult, not unreliable: there is no point in space both arms can reach, so no control strategy, no planner and no orientation mode can produce one. The task that would have demonstrated autonomy achieving something teleoperation cannot was blocked by geometry before it was blocked by anything else, and the finding is the reason it is not in the task set.

The cause is not the mount rotation, which was proved to leave the mirror residual invariant. It is that the two arms are parked asymmetrically at their home joint angles, and those angles are ground truth because the simulation to hardware bridge replays them onto the real arm. Changing them in software would command the difference as a jump. Fixing this requires re-parking the right arm in hardware.

Two of the five delivered tasks are the same bimanual mechanism with different objects, and only one task uses the second mechanism. There are two genuinely bimanual mechanisms available on this platform, not three. That is a real narrowing of what the platform can be used to ask.

### 8.4 Comparison against the planning report's targets

The delivered system diverges substantially from the plan, and the divergences fall into three groups.

*Superseded by measurement.* The planning report specified physical springs at the master for force feedback and virtual spring impedance at the slave. The impedance target is unreachable on this platform for a reason discovered during the work: the arms' cyclic control interface is unusable over the available network path, so the system commands through a high-level velocity interface at 18.4 to 18.7 Hz. An impedance law at 18 Hz is not an impedance law. The compliance the plan sought would have to come from the mechanism instead, through series elastic actuation at the joint [29, 15], or from a current-based formulation that does not require a high-rate torque loop [4]. Both are hardware changes.

*Deferred and still open.* Coordinated pick-and-place on the real platform was not demonstrated. The task set is specified and verified geometrically, and the runner exists, but the arms were not available for the period in which it would have been run.

*Added.* The safety architecture, the degradation architecture and the workspace characterisation were not in the plan. The first two are the direct consequence of discovering that sensor faults dominate; the third is the consequence of asking whether the task set was reachable before running it.

### 8.5 Threats to validity

#### 8.5.1 Simulation is not the arm

Every quantitative result in this thesis except the channel measurements comes from simulation with mock hardware. The mock echoes commanded positions with no dynamics, which has one consequence that must not be

glossed: every force reading is identically zero, so any quantity derived from a force is structurally undefined rather than approximated. The measurement that payload configuration made exactly 0 m of difference to tracking is a statement about the mock.

### 8.5.2 Latency figures are lower bounds

Every latency reported is control-path latency, measured between software stages. It excludes the operator's reaction, the display, and the physical arm's response. Any end-to-end figure will be larger, and the reported values should be read as a floor.

### 8.5.3 The instrument failure rate

Four times, a result that looked like a system failure was a defect in the measuring tool, and three of the four were caught only by checking the instrument against a known answer. The base rate is high enough that any unreproduced single measurement in this thesis should be treated as provisional. This is the reason the verification methodology exists and the reason the task-set figures are reported with their repeat count.

### 8.5.4 The one-stack constraint

Two instances of the master node split the serial stream between them and invalidated a full day of measurements before the cause was found. This is now prevented at the file-descriptor level, but it means that any measurement taken before that prevention was added carries a risk that cannot be retrospectively excluded.

## 8.6 What stands between this system and a two-person study

The study designed in [Chapter 5](#) cannot be run today, and the blockers are specific.

1. **Two master channels.** Left j2 and left j4. With both frozen the left arm has no radial dimension, so the direct teleoperation condition, which is the baseline the whole comparison rests on, is not representative of a working master.
2. **The right arm's home pose.** It has never been read from hardware, and the workspace geometry depends on it.
3. **Hardware validation of the safety layer.** Nothing in the safety architecture has run against a real arm. The Kortex session recovery path is the riskiest part, because the arm permits exactly one session and a leaked one refuses the next connection.
4. **Ethics approval for a two-person protocol**, which differs from a single-participant protocol in that one participant bears a physical risk arising from another participant's actions.

CONFIRM: current status of the ethics application with Dr. Zhang.

5. **A detection rate measured on real images.** The synthetic figure characterises the renderer.

None of these is a research question. All are execution, and the first is two soldered joints.

# Chapter 9

## Conclusion

This project built a teleoperation system for two seven-degree-of-freedom arms mounted on a worn backpack frame, commanded from an instrumented mannequin master, with a safety architecture appropriate to a manipulator standing beside a person and a shared-autonomy layer above it.

### 9.1 Against the objectives

*A master, built and characterised.* Fourteen potentiometer channels, two inertial sensors, two force sensors and two buttons on a single microcontroller, emitting a text frame at 100 Hz. Characterisation found six of fourteen channels usable and reversed two long-standing verdicts, both of which had come from computing statistics across transmitted rows rather than distinct sensor updates.

*A mapping that degrades predictably.* Position in spherical form with each component taken from the sensor that observes it best. Vertical and fore-and-aft motion track correctly on both arms. Lateral does not, and the measurements show why no axis map or recalibration can fix it. The cost of each channel reduction is quantified on 20 430 recorded frames, and the endpoint of the ladder reports position unavailable rather than inventing one.

*A safety architecture validated by injection.* Fourteen of fourteen injected faults handled. The two that took several attempts both failed for the same underlying reason, which is that a mechanism was keyed on the wrong observable: a dead-man on message arrival cannot detect a source that keeps publishing stale data.

*A workspace characterisation.* The two arms share no reachable workspace under the as-built mount, which blocks inter-arm handover for geometric reasons no software change can address. A buildable mount modification that would raise the shared cell count from three to twenty-two of sixty-three is quantified and deliberately not applied, because applying it invalidates the home pose, the workspace anchor and every clearance figure derived from them.

*A verification methodology.* Ten repeats over densified paths rather than single calls at endpoints, and an instrument validated against a known answer before a surprising measurement taken with it is reported. The five-task verification reproduced exactly across runs, which is the first reachability figure in this project to do so.

*Identification of what blocks progress.* Five items, listed in [Chapter 8](#). The first is two soldered joints.

### 9.2 What was not achieved

No result in this thesis was validated on a physical Kinova arm. The task set is verified geometrically and has never been performed. The two-person study the architecture was designed around was not run, and could not

have been: the direct teleoperation baseline depends on channels that are currently incoherent.

The planning report's haptic objective was not delivered and, as [Chapter 8](#) argues, is not deliverable on this platform through the available control interface.

### 9.3 Future work, in the order it should be done

1. **Repair left j2 and j4, and re-run the channel baseline after each attempt.** These two are named by the observability regression rather than chosen: with both frozen the left arm retains no reach observable at all, and repairing any other channel will not restore one. Re-running after each attempt rather than once at the end catches a repair that breaks a working channel.
2. **Correct the filter seed in the firmware** so the potentiometers do not spend 440 ms settling from a railed value after every board reset. The change is one line and does not alter the filter's steady state.
3. **Read the right arm's home joint angles from hardware.** They have never been read, and the workspace geometry depends on them.
4. **Bring up a second inertial sensor per arm on the secondary bus.** The elbow angle then comes from the relative orientation of two segments with no potentiometer, and the shared yaw ambiguity cancels in a relative measurement, which is why two sensors can replace the elbow and one cannot. It costs two potentiometer channels and reaches 63.8 mm on a single remaining pot.
5. **Re-park the right arm and re-derive the mount.** This is the only route to a shared workspace, and therefore the only route to inter-arm handover and to a wider bimanual task set. It should be one deliberate pass, because the home pose, the anchor and every clearance figure move together.
6. **Validate the safety layer on hardware**, starting with the session recovery path, which is the riskiest because the arm permits exactly one session.
7. **Run the two-person study.** The exchange rate between operator performance and wearer trust is the open question this architecture was built to ask, and it remains open.

### 9.4 Closing

The most transferable outcome of this work is not the mapping or the safety architecture. It is the discipline that produced them. Four times a result that looked like a system failure turned out to be a defect in the measuring tool, and each was caught only by checking the instrument against something whose answer was already known. On a robot that stands beside a person, the difference between a measurement and an artefact is not an academic distinction, and the habit of establishing which one is in hand before acting on it is the part of this project most worth carrying forward.

# Chapter A

## Master arm firmware

The complete sketch is at `firmware/master_arm/teensy_final.ino`. The superseded revision, which carried the pin conflict described in [Chapter 3](#), is kept at `firmware/master_arm/teensy_merged_prior.ino` so the change can be diffed.

**CONFIRM:** `MPU6050.h` is included by the sketch but was not among the files handed over and is not in the repository. It must be attached here before submission.

### A.1 Pin assignment as declared

Listing A.1: Pin arrays. A4 and A5 appear in neither array, which is the fix described in [Chapter 3](#).

```
1  const int potPinsK1[JOINTS] =
2  {
3    A0, A1, A2, A3, A6, A7, A8      // K1 (left arm) -- 7 channels, no I2C pins
4  };
5  const int potPinsK2[JOINTS] =
6  {
7    A9, A10, A11, A12, A13, A16, A17 // K2 (right arm) -- 7 channels, no I2C pins
8  };
9
10 const int FSR1_PIN = A14;
11 const int FSR2_PIN = A15;
12
13 const int BUTTON1_PIN = 2;
14 const int BUTTON2_PIN = 3;
```

### A.2 Signal conditioning as implemented

Listing A.2: The potentiometer filter and the count-to-degree transform. The filter operates in raw ADC units; note that `setup()` seeds the accumulator with `analogRead(pin) << 8`, which is inconsistent with this recurrence and produces the start-up transient quantified in [Table 3.3](#).

```
1  static constexpr int ALPHA_FP    = 64;
2  static constexpr int ADC_MIN_VAL = 50;
3  static constexpr int ADC_MAX_VAL = 4045;
4
```

```

5 inline int updateEMA(int pin, int32_t& emaFP)
6 {
7     int raw = analogRead(pin);
8     emaFP =
9     ((int32_t)ALPHA_FP * raw +
10     (256 - ALPHA_FP) * emaFP) >> 8;
11     return (int)emaFP;
12 }
13
14 inline float adcToDeg(int adc)
15 {
16     float t = constrain(
17     (float)(adc - ADC_MIN_VAL) /
18     (float)(ADC_MAX_VAL - ADC_MIN_VAL),
19     0.0f,
20     1.0f);
21     return t * DEG_MAX;
22 }

```

Listing A.3: Button debounce and toggle. The output is a latched state, so a host that drops frames cannot miss a press.

```

1 static constexpr uint32_t DEBOUNCE_MS = 40;
2
3 inline int updateButtonToggle(
4 int pin, int& state, int& lastRaw, uint32_t& lastChangeMs)
5 {
6     int raw = digitalRead(pin);
7     uint32_t now = millis();
8     if(raw != lastRaw && (now - lastChangeMs) > DEBOUNCE_MS)
9     {
10         lastChangeMs = now;
11         lastRaw = raw;
12         if(raw == LOW)
13         {
14             state = state ? 0 : 1;
15         }
16     }
17     return state;
18 }

```

## A.3 Frame construction

Listing A.4: Frame assembly. Inertial values are transmitted raw in LSB, so the scale factors are applied on the host and can be corrected without reflashing.

```

1     for(int j=0;j<JOINTS;j++)
2         p += snprintf(p, end-p, "k1j%d:%.1f,", j+1, degK1[j]);
3     for(int j=0;j<JOINTS;j++)
4         p += snprintf(p, end-p, "k2j%d:%.1f,", j+1, degK2[j]);
5     p += snprintf(p, end-p, "K1IMU:%d,%d,%d,%d,%d,%d,", ax1,ay1,az1,gx1,gy1,gz1);
6     p += snprintf(p, end-p, "K2IMU:%d,%d,%d,%d,%d,%d,", ax2,ay2,az2,gx2,gy2,gz2);
7     p += snprintf(p, end-p, "fsr1:%d,fsr2:%d,", fsr1, fsr2);

```

```
8 | p += snprintf(p, end-p, "btn1:%d,btn2:%d", btn1, btn2);
```

# Chapter B

## Supporting records

The following are maintained in the repository rather than reproduced here, because each is longer than the chapter that cites it and each is generated rather than written.

Table B.1: Supporting records and where they live.

| Record                      | Content                                                                                       |
|-----------------------------|-----------------------------------------------------------------------------------------------|
| thesis_report/DIVERGENCE.md | Divergence from the planning report, by objective, with the reason and the evidence for each. |
| thesis_report/EVIDENCE.md   | Every numerical claim in this thesis with the file and run that produced it.                  |
| recordings/baselines/       | Channel-health baselines by date, workspace sweeps, and the five-task verification result.    |
| docs/NEXT_SESSION.md        | Current blocking items in priority order.                                                     |

# Bibliography

- [1] Patrick Beeson and Barrett Ames. TRAC-IK: An open-source library for improved solving of generic inverse kinematics. In *Proc. IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*, pages 928–935. IEEE, 2015.
- [2] Gunnar A. Borg. Psychophysical bases of perceived exertion. *Medicine and Science in Sports and Exercise*, 14(5):377–381, 1982.
- [3] Matthew Botvinick and Jonathan Cohen. Rubber hands ‘feel’ touch that eyes see. *Nature*, 391(6669):756, 1998.
- [4] S. de Wolde, J. Kober, and R. Babuska. Current-based impedance control for interacting with mobile manipulators. *arXiv preprint arXiv:2403.13079*, 2024.
- [5] Anca D. Dragan and Siddhartha S. Srinivasa. A policy-blending formalism for shared control. *The International Journal of Robotics Research*, 32(7):790–805, 2013.
- [6] Paul M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6):381–391, 1954.
- [7] Jacob W. Guggenheim and H. Harry Asada. Inherent haptic feedback from supernumerary robotic limbs. *IEEE Transactions on Haptics*, 14(1):123–131, 2021.
- [8] Sandra G. Hart and Lowell E. Staveland. Development of NASA-TLX (task load index): Results of empirical and theoretical research. In Peter A. Hancock and Najmedin Meshkati, editors, *Human Mental Workload*, volume 52 of *Advances in Psychology*, pages 139–183. North-Holland, 1988.
- [9] Imperial College London. MUlti-limb Virtual Environment (Dr. Octopus). <https://www.imperial.ac.uk/a-z-research/human-robotics/dr-octopus/>. Accessed April 2026.
- [10] International Society of Automation. *ANSI/ISA-101.01-2015: Human Machine Interfaces for Process Automation Systems*, 2015.
- [11] InvenSense Inc. *MPU-6000 and MPU-6050 Product Specification, Revision 3.4*, 2013. Document PS-MPU-6000A-00.
- [12] InvenSense Inc. *MPU-6000 and MPU-6050 Register Map and Descriptions, Revision 4.2*, 2013. Document RM-MPU-6000A-00.
- [13] Shervin Javdani, Siddhartha S. Srinivasa, and J. Andrew Bagnell. Shared autonomy via hindsight optimization. In *Robotics: Science and Systems XI*, 2015.
- [14] Kinova Robotics. *Gen3 Ultra Lightweight Robot User Guide*, 2022.

- [15] C. Lee, S. Kwak, and J. Kwak. Development, analysis, and control of series elastic actuator for a wearable robotic leg. *Frontiers in Neurorobotics*, 13:17, 2019.
- [16] Matthew R. Longo, Friederike Schüür, Marjolein P. M. Kammers, Manos Tsakiris, and Patrick Haggard. What is embodiment? a psychometric approach. *Cognition*, 107(3):978–998, 2008.
- [17] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.
- [18] C. Moon and J. Kim. Assessing the physical impact of supernumerary limbs on a human subject: A simulation study. In *Proc. 46th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)*, pages 1–4. IEEE, 2024.
- [19] NVIDIA. Isaac lab documentation: Impedance control and motion generators. <https://isaac-sim.github.io/IsaacLab>, 2026. Accessed April 2026.
- [20] Federico Parietti and H. Harry Asada. Dynamic analysis and state estimation for wearable robotic limbs subject to human-induced disturbances. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 3880–3887. IEEE, 2013.
- [21] Federico Parietti and H. Harry Asada. Supernumerary robotic limbs for aircraft fuselage assembly: Body stabilization and guidance by bracing. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 1176–1183. IEEE, 2014.
- [22] Federico Parietti and H. Harry Asada. Supernumerary robotic limbs for human body support. *IEEE Transactions on Robotics*, 32(2):301–311, 2016.
- [23] Mattia Pinardi, Matthew R. Longo, Domenico Formica, Matija Strbac, Carsten Mehring, Etienne Burdet, and Giovanni Di Pino. Impact of supplementary sensory feedback on the control and embodiment in human movement augmentation. *Communications Engineering*, 2:64, 2023.
- [24] PJRC. Teensy 4.1 development board. <https://www.pjrc.com/store/teensy41.html>. Pinout reference card rev. 4. Accessed August 2026.
- [25] Domenico Prattichizzo et al. Human augmentation by wearable supernumerary robotic limbs: review and perspectives. *Progress in Biomedical Engineering*, 2021.
- [26] MHD Yamen Saraiji, Tomoya Sasaki, Reo Matsumura, Kouta Minamizawa, and Masahiko Inami. Fusion: full body surrogacy for collaborative communication. In *ACM SIGGRAPH 2018 Emerging Technologies*, pages 1–2. ACM, 2018.
- [27] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033. IEEE, 2012.
- [28] Yuchen Tong and Jianbin Liu. Review of research and development of supernumerary robotic limbs. *IEEE/CAA Journal of Automatica Sinica*, 2021.
- [29] H. Toubar, G. Goldsmith, S. A. Tafrishi, M. Innala, and V. Bergstromer. Design, modeling, and control of a series elastic actuator with discretely adjustable stiffness. *Mechanism and Machine Theory*, 170:104720, 2022.

- [30] Bo Yang, Jian Huang, Xu Chen, Caihua Xiong, and Yasuhisa Hasegawa. Supernumerary robotic limbs: A review and future outlook. *IEEE Transactions on Medical Robotics and Bionics*, 3(3):623–639, 2021.
- [31] K. Zhang et al. Review of supernumerary robotic limbs. *Journal of Physics: Conference Series*, 2023.
- [32] Y. Zhang et al. Motion-compensation control of supernumerary robotic arms subject to human-induced disturbances. *Advanced Intelligent Systems*, 2024.
- [33] Y. Zheng et al. Supernumerary robotic limbs: A quantitative analysis and review of design, embodiment, and future challenges. *Journal of Field Robotics*, 2026.
- [34] Haoyuan Zhou, Anzhu Gao, Pierre Ducos, Jonathan Eden, and Etienne Burdet. Spatial guidelines for supernumerary robotic limbs in near-body interactions. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. ACM, 2026.